

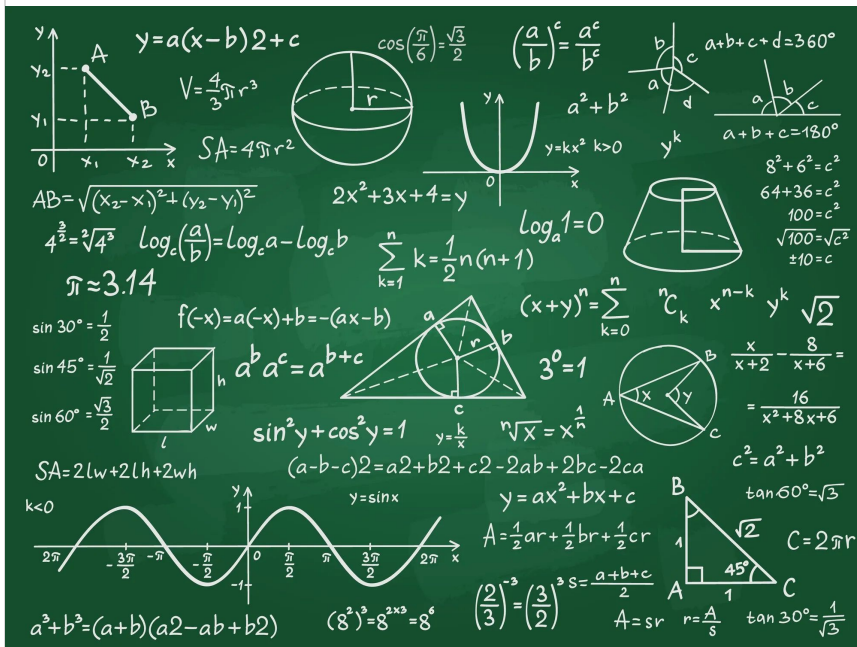


Game Math - Advanced

Matematica per Game Programmers



Programma corso

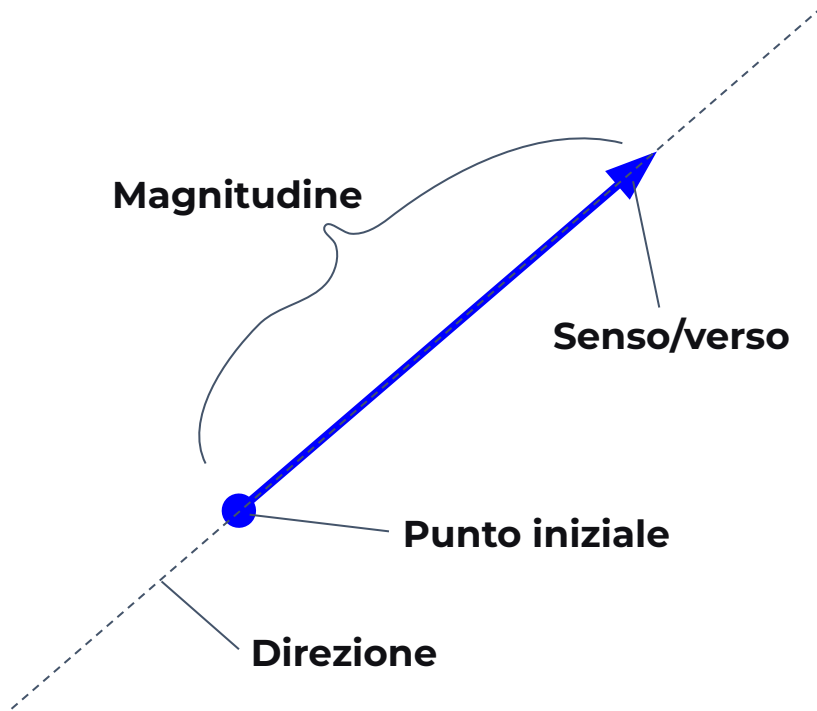


- Ripasso Vettori
 - Dot Product
 - Cross Product
- Interpolazione lineare
- Trigonometria
 - Seno e coseno
 - Coordinate polari
- Quaternioni
- Transform e trasformazioni
- Curve di Bézier
 - Semplici
 - Complesse
- Simulazione fisica
 - Cinematica
 - Balistica
 - Bersaglio mobile, no gravità
 - Bersaglio fermo, con gravità



Vettori

DIREZIONE E MAGNITUDINE



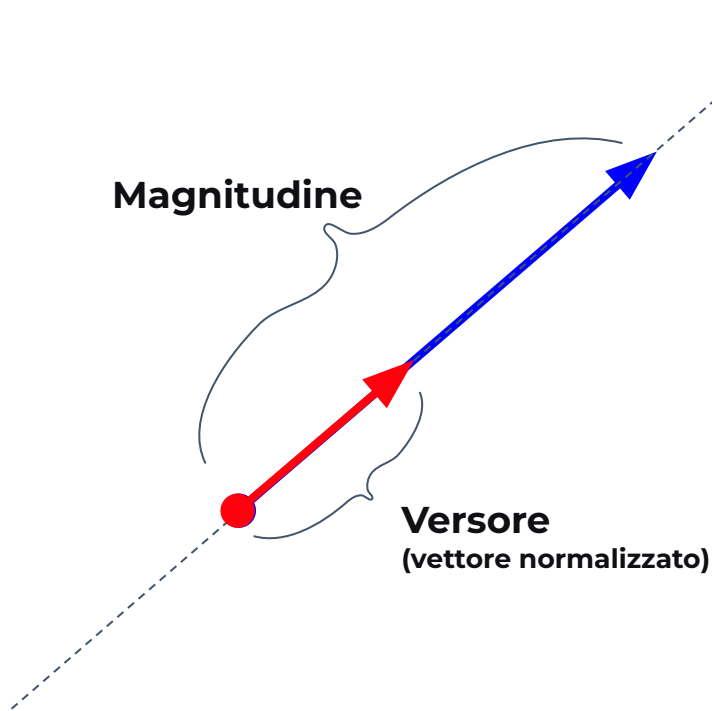
Un vettore è la rappresentazione matematica di uno **spostamento** ed è composto dalle seguenti caratteristiche base:

1. **Punto iniziale(initial point):** punto di **origine** del vettore.
Quando un vettore definisce una posizione, il punto iniziale è 0,0,0
2. **Direzione (direction):** **retta** sulla quale passa il vettore
3. **Senso/verso (sense):** **senso** verso il quale il vettore è rivolto, lungo la sua direzione.
Due vettori possono avere **stessa direzione ma verso opposto**.
4. **Magnitude (magnitudine):** **lunghezza** del vettore, a volte chiamata anche *intensità*



Vettori

DIREZIONE E MAGNITUDINE



Un vettore è **normalizzato** quando la sua lunghezza/magnitudine è **1**. Per avere un vettore normalizzato basta dividerlo per la sua lunghezza.

$$|v| = \sqrt{v_1^2 + v_2^2 + v_3^2 + \dots + v_n^2}$$

Nel piano cartesiano 2D

$$|v| = \sqrt{x_v^2 + y_v^2}$$

Nello spazio 3D

$$|v| = \sqrt{x_v^2 + y_v^2 + z_v^2}$$

Attenzione alla radice quadrata!



Vettori

CALCOLO VETTORIALE

Somma vettoriale

Primo vettore $\Rightarrow \mathbf{a} (x_a, y_a, z_a)$

Secondo vettore $\Rightarrow \mathbf{b} (x_b, y_b, z_b)$

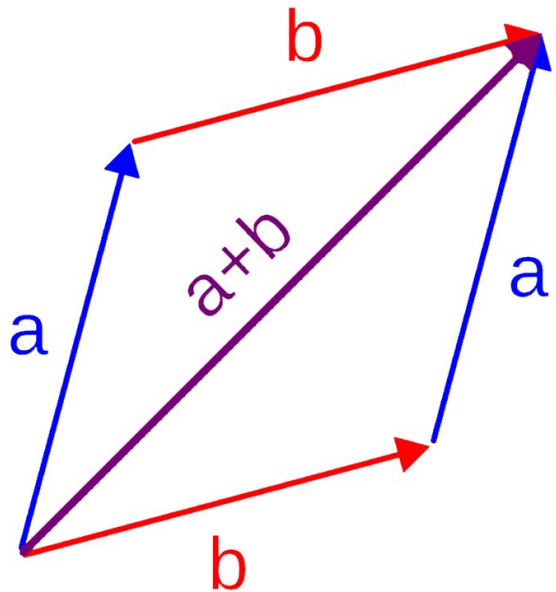
Somma delle **componenti** dei due vettori

$$\mathbf{a} + \mathbf{b} = (x_a + x_b, y_a + y_b, z_a + z_b)$$

L'ordine delle operazioni non cambia il risultato

$$\mathbf{a} + \mathbf{b} = \mathbf{b} + \mathbf{a}$$

(proprietà commutativa)





Vettori

CALCOLO VETTORIALE

Sottrazione vettoriale (regola del triangolo)

Primo vettore $\Rightarrow \mathbf{a} (x_a, y_a, z_a)$

Secondo vettore $\Rightarrow \mathbf{b} (x_b, y_b, z_b)$

Sottrazione delle **componenti** dei due vettori

$$\mathbf{c} = \mathbf{a} - \mathbf{b} = (x_a - x_b, y_a - y_b, z_a - z_b)$$

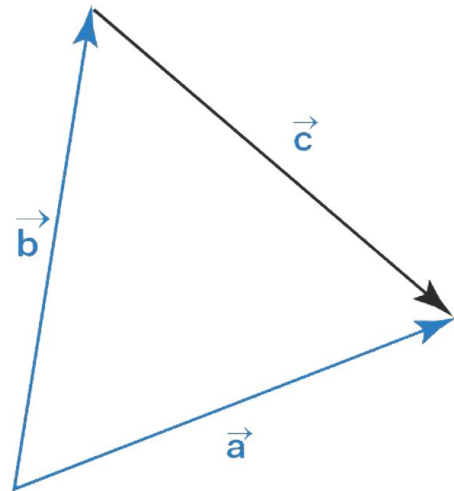
L'ordine delle operazioni **cambia** il risultato

$$\mathbf{a} - \mathbf{b} \neq \mathbf{b} - \mathbf{a}$$

Utilizzata per trovare il vettore tra due punti

Vettore **da A a B** $= \mathbf{B} - \mathbf{A}$

ovvero **punto finale - punto iniziale**





Vettori

CALCOLO VETTORIALE

Moltiplicazione scalare

Vettore => $\mathbf{a} (x_a, y_a, z_a)$

Scalare => 3

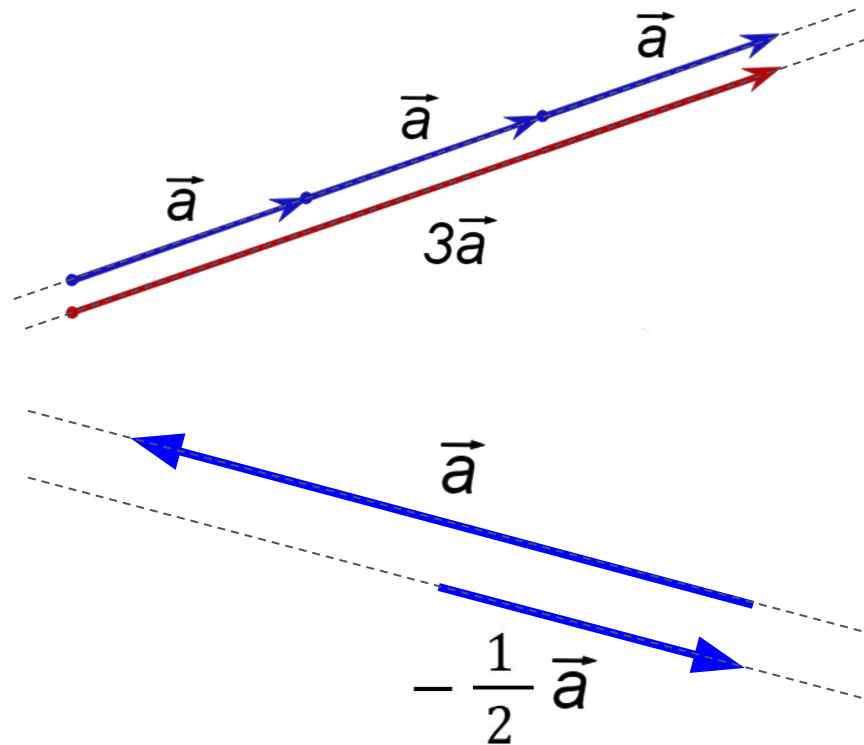
Sottrazione delle **componenti** dei due vettori

$$3\mathbf{a} = (3x_a, 3y_a, 3z_a)$$

L'ordine delle operazioni **non cambia** il risultato

$$3 * \mathbf{a} = \mathbf{a} * 3$$

La moltiplicazione scalare **non cambia la direzione** ma **può cambiare il verso** quando lo scalare è **negativo**





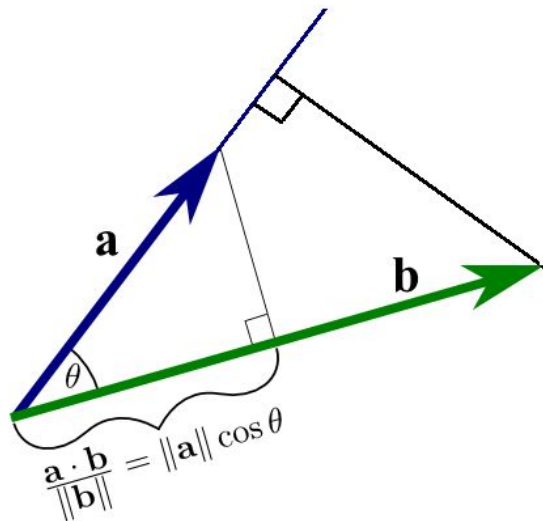
Vettori

PRODOTTO SCALARE - DOT PRODUCT

Vettore * Vettore = Scalare

$$\vec{a} \cdot \vec{b} = |\vec{a}| \cdot |\vec{b}| \cdot \cos(\theta)$$

Il prodotto scalare tra due vettori equivale alla **proiezione** del primo vettore sul secondo vettore moltiplicata per la lunghezza del secondo vettore



Quando i vettori dell'operazione sono **normalizzati** il risultato indica il coseno dell'angolo compreso tra i due vettori

L'**arcocoseno (acos)** della lunghezza del vettore risultante ci restituisce l'angolo compreso tra i due vettori

$$\vec{a} \cdot \vec{b} = x_a x_b + y_a y_b + z_a z_b$$



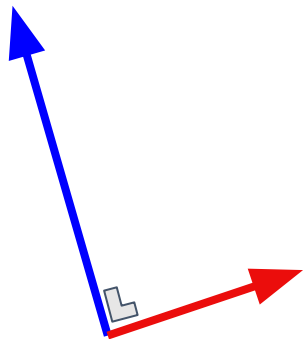
Vettori

PRODOTTO SCALARE - DOT PRODUCT

Vettori **perpendicolari** hanno prodotto scalare pari a **0** → **$\text{Acos}(0) = 90^\circ$**

Vettori normalizzati **paralleli** hanno prodotto scalare pari a

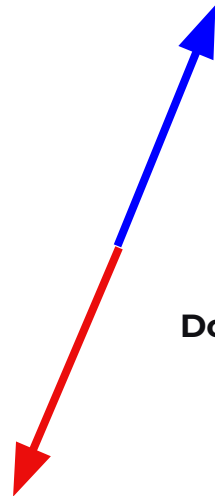
1. **1** (stesso verso) → **$\text{Acos}(1) = 0^\circ$**
2. **-1** (verso opposto) → **$\text{Acos}(-1) = 180^\circ$**



Dot = 0



Dot = 1



Dot = -1



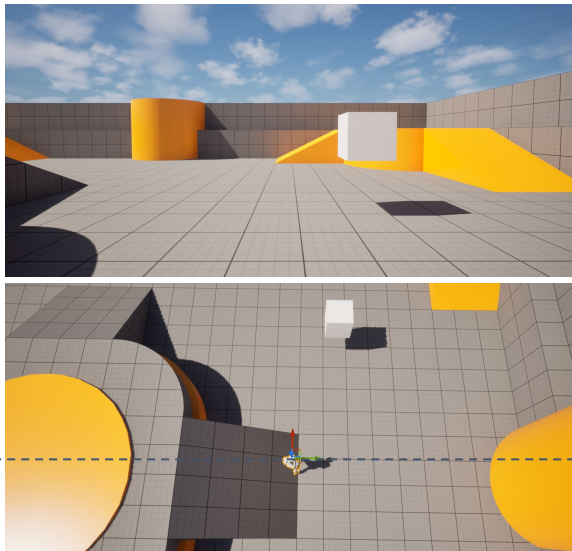
Vettori

PRODOTTO SCALARE - ESERCIZI

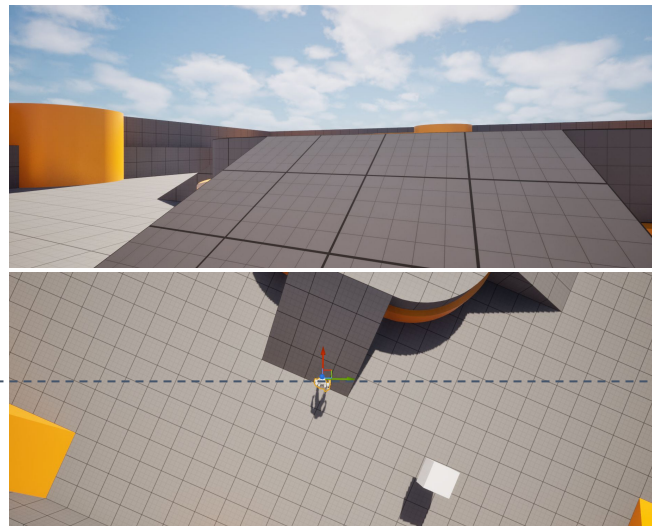
Single Facing (2 casi)

1. Progetto **FPS/TPS** template
2. Aggiungere **oggetto statico** nel livello
3. Stampare a schermo la **posizione relativa alla telecamera dell'oggetto** per i seguenti 2 casi: **davanti e dietro**

DAVANTI



DIETRO



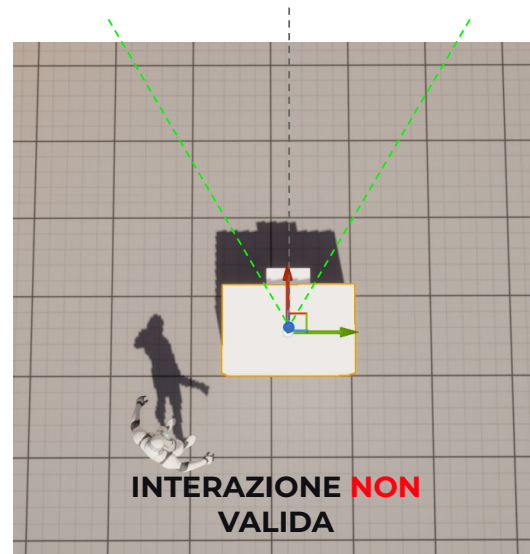
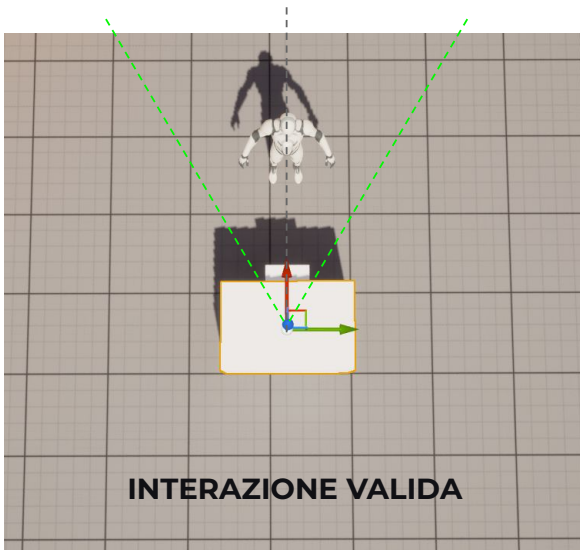


Vettori

PRODOTTO SCALARE - ESERCIZI

Single Facing (angolo)

1. Progetto **TPS** template
2. Aggiungere **oggetto interagibile con un verso** - (esempio: baule) - con un angolo di interazione tunabile
3. **Permettere l'interazione** solo se il player si trova nell'angolo di interazione - **ignorando** l'orientamento del player



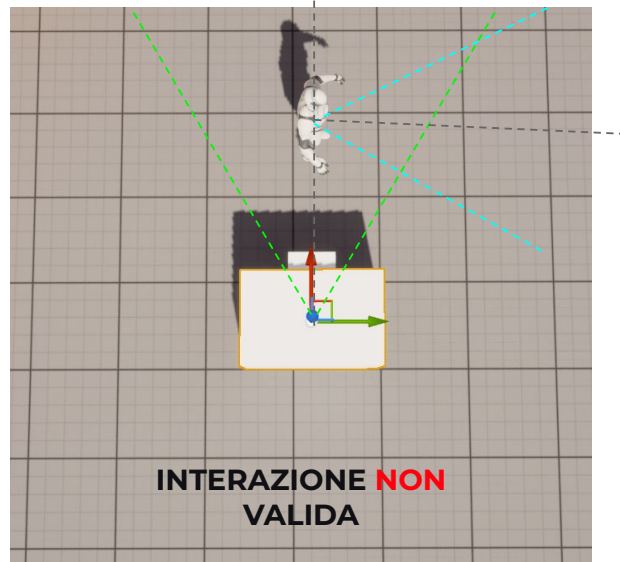
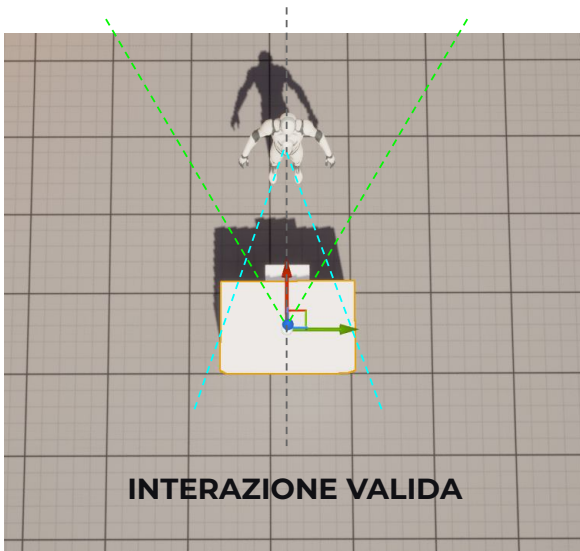


Vettori

PRODOTTO SCALARE - ESERCIZI

Single Facing (angoli)

1. Progetto **TPS** template
2. Aggiungere **oggetto interagibile con un verso** - (esempio: baule) - con un angolo di interazione tunabile
3. **Permettere l'interazione** solo se il player si trova nell'angolo di interazione e sta guardando l'oggetto **entro un altro angolo tunabile**





Vettori

PRODOTTO SCALARE - ESERCIZI

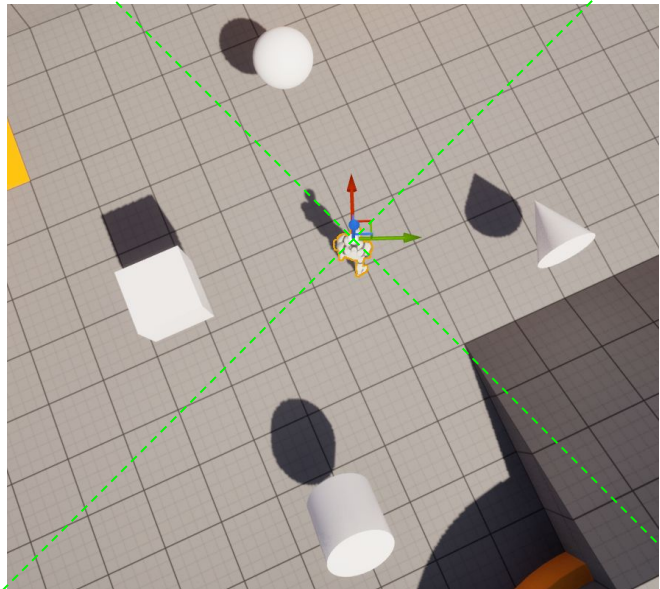
Single Facing (4 casi)

1. Progetto **TPS** template
2. Aggiungere **oggetto statico** nel livello
3. Stampare a schermo la **posizione relativa dell'oggetto** per i 4 casi:
davanti, dietro, sinistra e destra

Nell'immagine a destra gli oggetti vengono considerati nelle seguenti posizioni relative:

1. **Davanti:** sfera
2. **Dietro:** cilindro
3. **Sinistra:** cubo
4. **Destra:** cono

Gli angoli da considerare sono tutti a **45°** rispetto ai vettori **forward/back/left/right** del player pawn



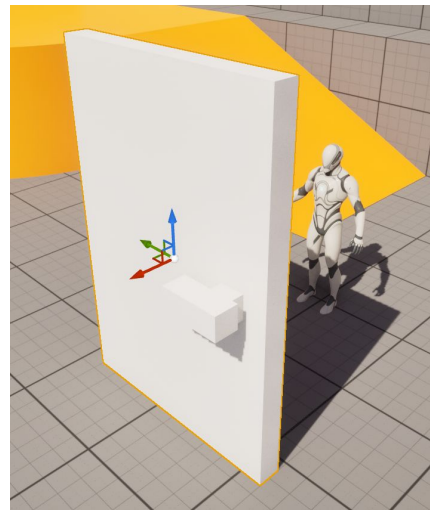
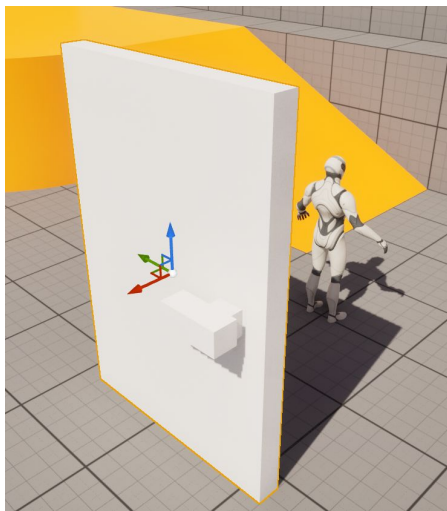
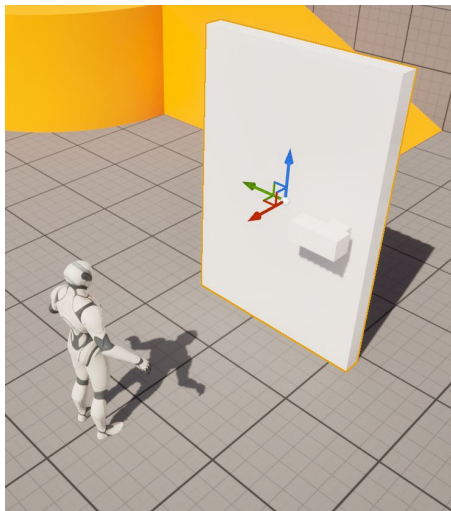


Vettori

PRODOTTO SCALARE - ESERCIZI

Double facing

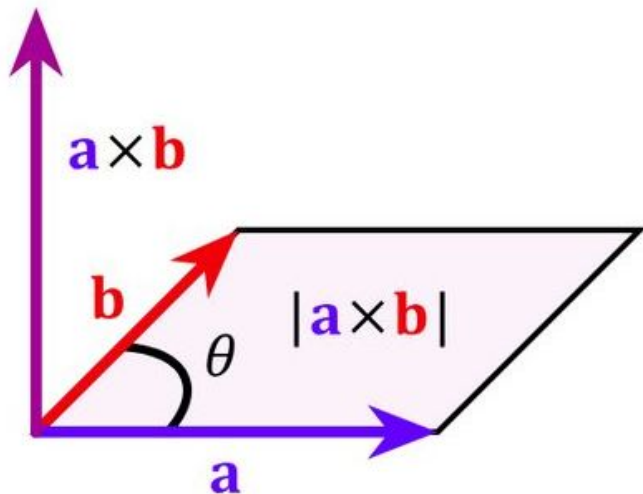
1. Progetto **TPS** template
2. Aggiungere **oggetto interagibile con un verso** - porta
3. Stampare a schermo i seguenti casi:
 - a. Interagibile e player **si stanno guardando**
 - b. Interagibile e player **guardano direzioni opposte**
 - c. **Solo uno** dei due **guarda** l'altro (dicendo quale)





Vettori

PRODOTTO VETTORIALE - CROSS PRODUCT



Vettore $\wedge$ Vettore = Vettore

$$\overline{a} \wedge \overline{b} = |\overline{a}| \cdot |\overline{b}| \cdot \sin(\theta) \cdot \overline{n}$$

Il prodotto vettoriale tra due vettori equivale ad un **vettore perpendicolare al piano** formato dai due vettori dell'operazione.

La lunghezza del vettore risultante è pari all'area del *parallelogramma* creato dai due vettori

Quando i vettori dell'operazione sono **normalizzati** il vettore risultante è lungo quanto il *seno dell'angolo compreso* tra i due vettori,

L'**arcoseno (asin)** della lunghezza del vettore risultante ci restituisce l'angolo compreso tra i due vettori



Vettori

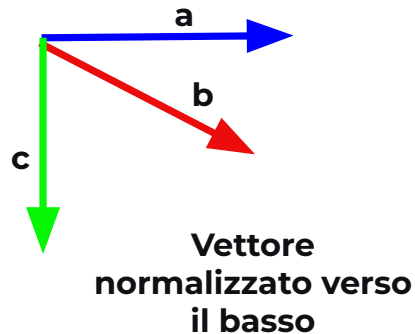
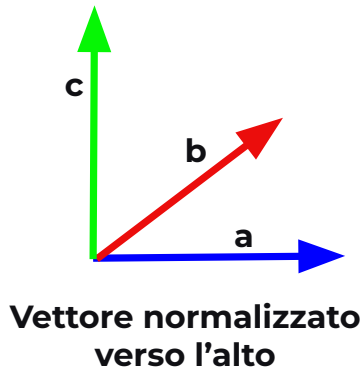
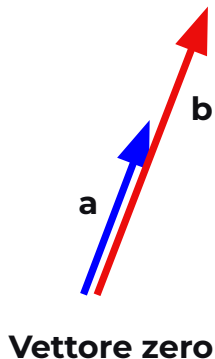
PRODOTTO VETTORIALE - CROSS PRODUCT

Vettori **paralleli** hanno prodotto vettoriale pari al **vettore zero**

Vettori normalizzati **perpendicolari** hanno prodotto vettoriale pari ad un vettore lungo 1

1. Verso l'alto se il secondo vettore è a sinistra rispetto al primo
2. Verso il basso se il secondo vettore è a destra rispetto al primo

Quando i due vettori iniziali sono sul piano **XY** il segno della componente **Z del vettore risultante** ci indica la **posizione relativa (sinistra/destra)** del secondo vettore rispetto al primo





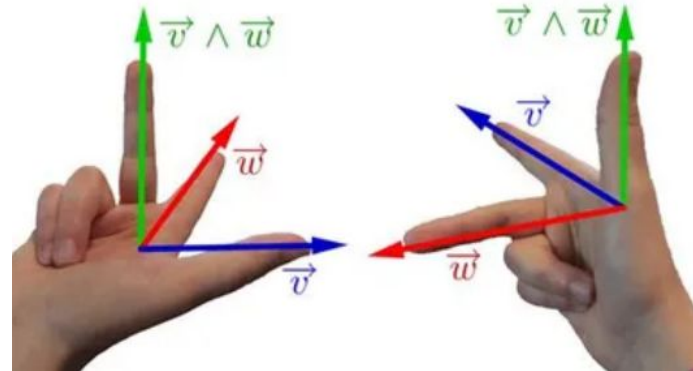
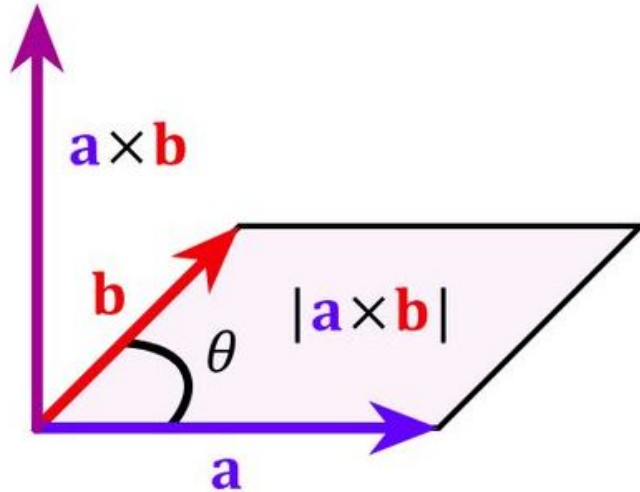
Vettori

PRODOTTO VETTORIALE - CROSS PRODUCT

Direzione e **verso** del vettore risultante dipendono dalla **regola della mano destra**

Può essere utilizzata in due modi

1. **Pollice** $\wedge$ **Indice** = **Medio**
2. **Indice** $\wedge$ **Medio** = **Pollice**





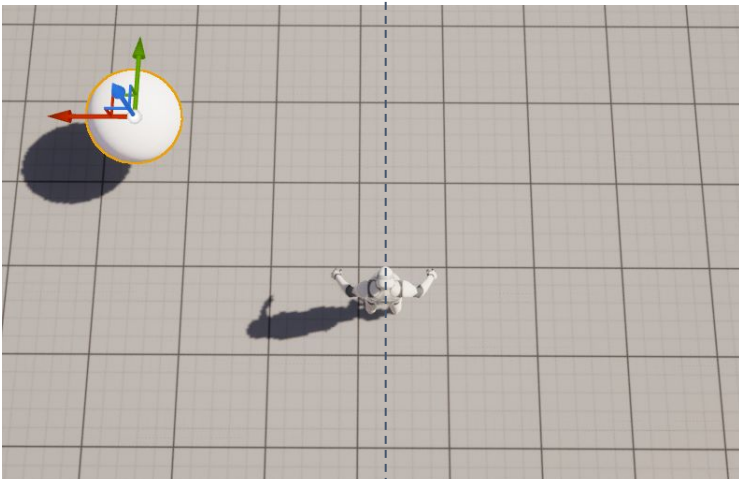
Vettori

PRODOTTO VETTORIALE - ESERCIZI

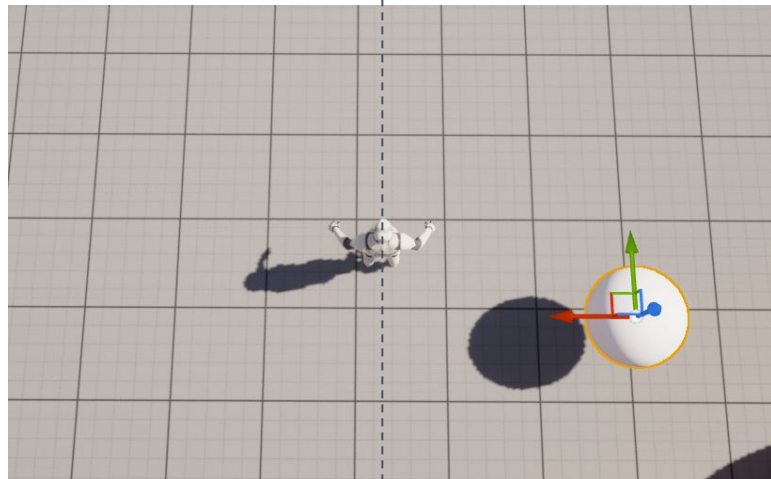
Side detection (sinistra/destra)

1. Progetto **TPS** template
2. Aggiungere **oggetto statico** nel livello
3. Usando il **cross product**, stampare a schermo la **posizione relativa dell'oggetto** per i 2 casi: **sinistra o destra**
4. Usare il **dot product** per ottenere lo stesso risultato

SINISTRA



DESTRA





Vettori

DOT vs CROSS vs ATAN

Dot product = $\cos$ angolo relativo tra vettori



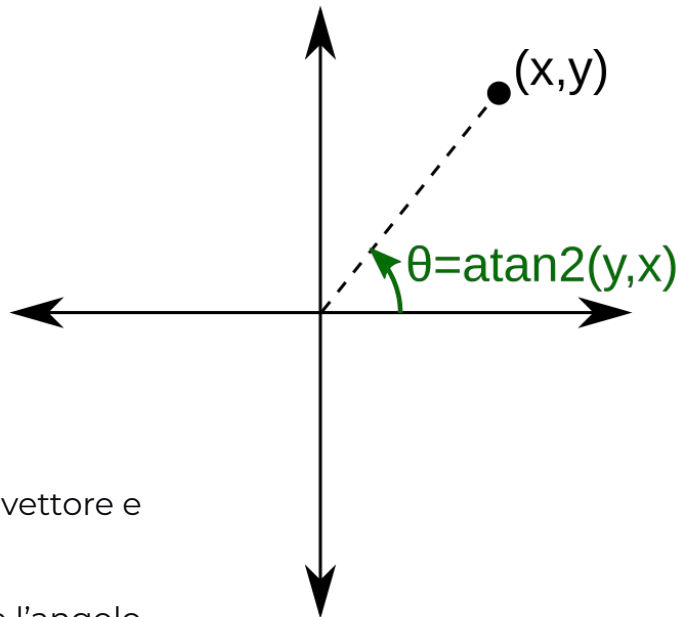
Cross product = $\sin$ angolo relativo tra vettori



Arcotangente = angolo assoluto tra un qualsiasi vettore e il vettore dell'asse x (1,0)

Fare l'arcotangente di una direzione ci restituisce l'angolo **con segno** tra l'asse x e quella direzione

Atan2(y,x) equivale a **Atan**(y/x) con il controllo su $x \neq 0$





Vettori

ATAN - ESERCIZI

Bussola

1. Progetto **FPS** template
2. Stampare a schermo il punto cardinale che si sta guardando (Nord, Nord-Ovest, Ovest, etc...) utilizzando l'arcotangente



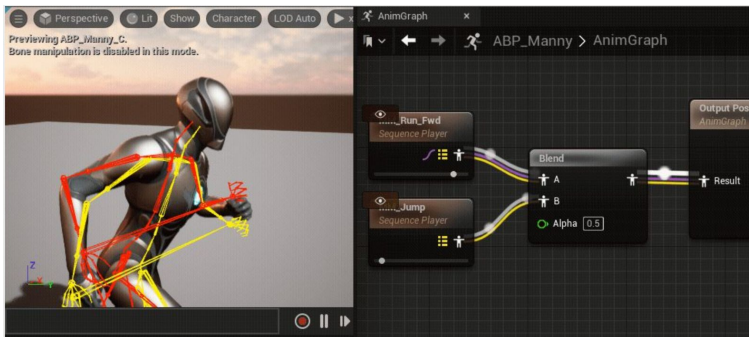
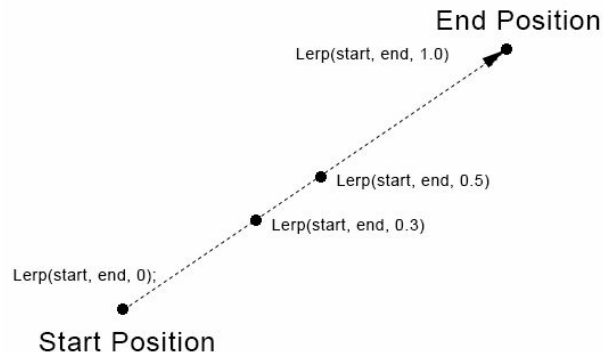


Interpolazione lineare

Interpolazione lineare (LERP) è una funzione lineare che serve a vari scopi:

1. Muovere un **valore interpolato C** dal valore iniziale **A** al valore finale **B** in modo controllato, mediante **parametro di interpolazione t**
2. Trovare un punto intermedio **C pesato** tra due punti **A** e **B**
3. Blend di animazioni

$$C = (B - A) * t + A$$



Nelle animazioni l'interpolazione lineare viene applicata posizionando ogni parte del corpo del character (bones) in una posizione **interpolata** tra le posizioni che avrebbe ciascuna parte del corpo in ogni animazione.

(**Alpha** è il parametro di interpolazione **t**)



Interpolazione lineare

ESERCIZI

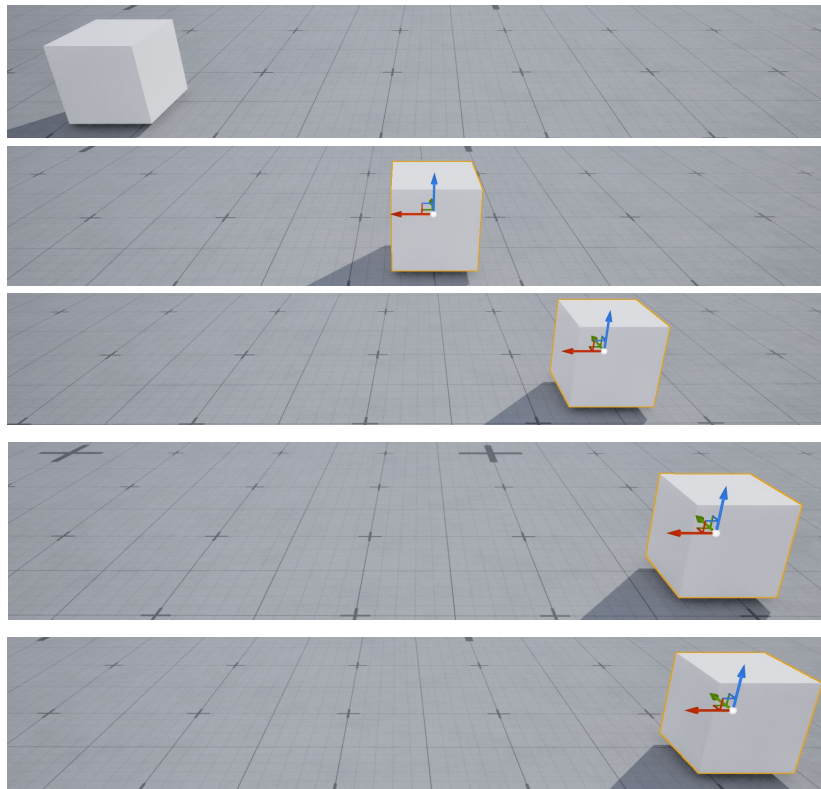
Consideriamo un movimento **tra due punti tunabili**

Movimento smorzato

1. Muovere un actor *dal punto iniziale al punto finale* in modo interpolato considerando di smorzare il movimento man mano che l'oggetto si avvicina al *punto finale*.
2. Rendere tunabile la **reattività**.

Smorzamento

Posizione corrente =
 $\text{Lerp}(\text{Posizione corrente},$
 $\text{Posizione da raggiungere},$
 $\text{Reattività});$





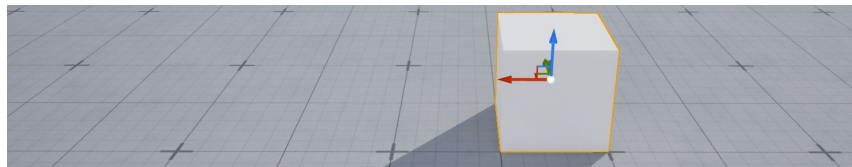
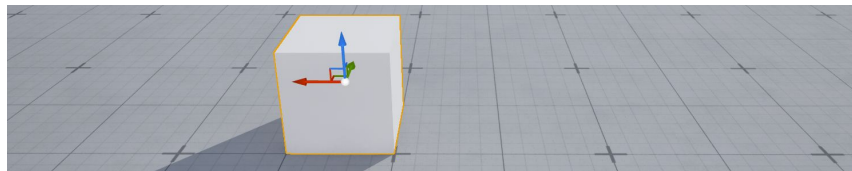
Interpolazione lineare

ESERCIZI

Consideriamo un movimento **tra due punti tunabili**

Movimento a tempo costante

1. Muovere un actor *dal punto iniziale al punto finale* in modo interpolato in un *intervallo di tempo prestabilito e tunabile*
2. Far tornare **indietro** l'actor al punto iniziale quando raggiunge il punto finale
3. **Invertire il movimento** alla pressione di un tasto a scelta, senza cambiare l'intensità della velocità di moto
 - a. Esempio: quando l'oggetto è a *metà strada*, premo l'input scelto e l'oggetto torna indietro, impiegando solo *metà* del tempo essendo la velocità costante





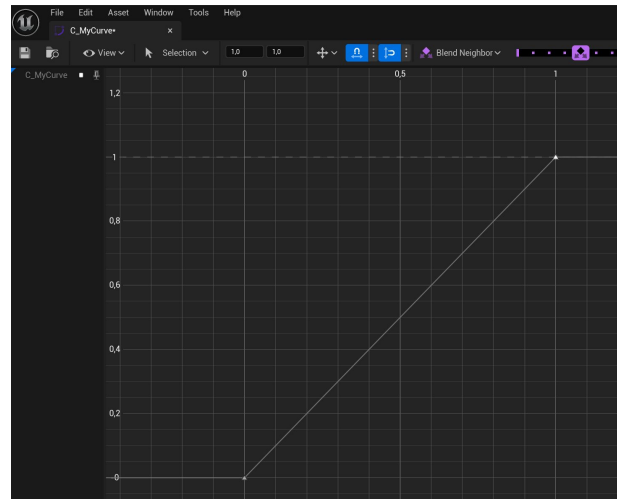
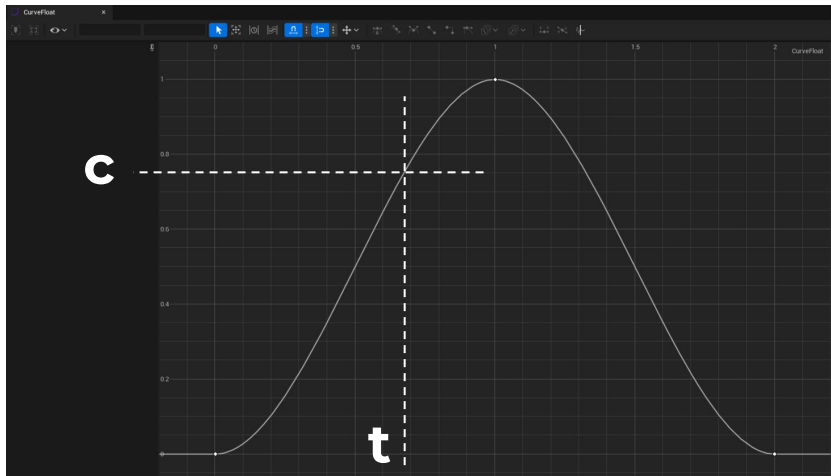
Interpolazione lineare

CURVE

Le curve di Unreal sono utili per gestire l'andamento di un'interpolazione lineare (**CurveFloat**)

Invece di usare un parametro t lineare (ad esempio il tempo percentuale/normalizzato) usiamo un parametro **restituito dalla curva**

float c = Curve→GetFloatValue(t)



Una “curva” **lineare** che va da (0,0) a (1,1) equivale ad usare direttamente t come *parametro di interpolazione*



Interpolazione lineare

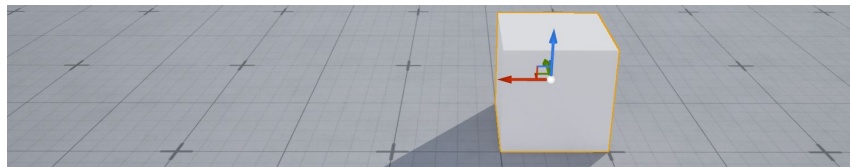
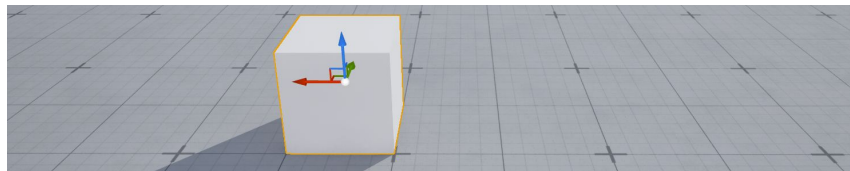
ESERCIZI

Consideriamo un movimento **tra due punti tunabili**

Movimento a tempo costante con curva

1. Muovere un actor *dal punto iniziale al punto finale* in modo interpolato in un *intervallo di tempo tunabile* **seguendo una curva tunabile**
2. Far tornare **indietro** l'actor al punto iniziale quando raggiunge il punto finale
3. **Invertire il movimento** alla pressione di un tasto a scelta, senza cambiare l'intensità della velocità di moto o il percorso effettuato

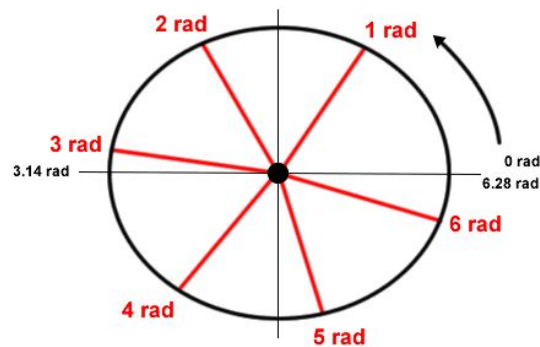
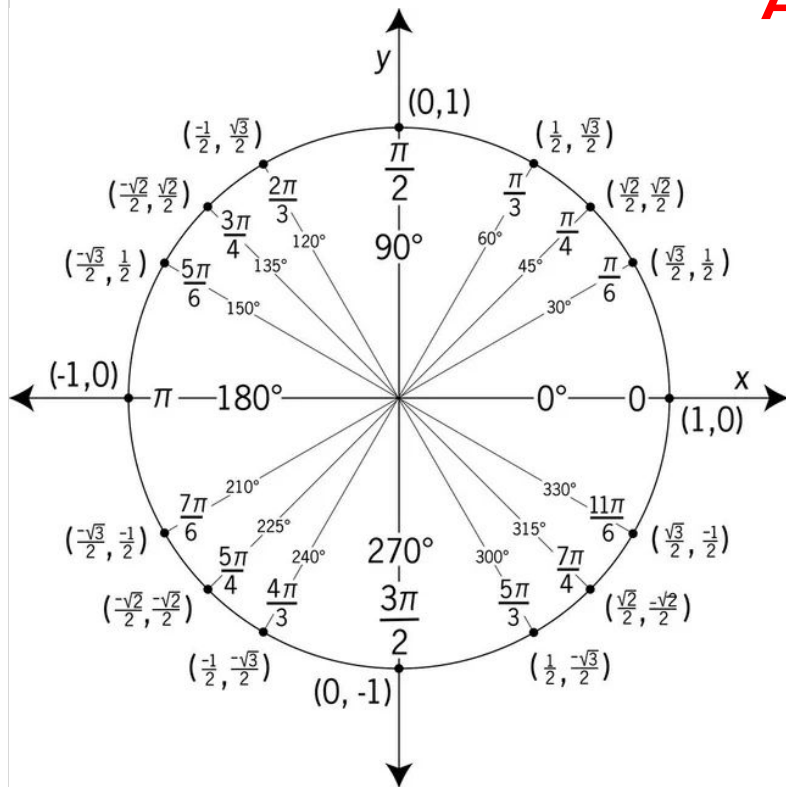
La curva **deve** iniziare a (0,0) e finire a (1,1) per poter raggiungere rispettivamente il punto iniziale e il punto finale





Trigonometria

ANGOLI



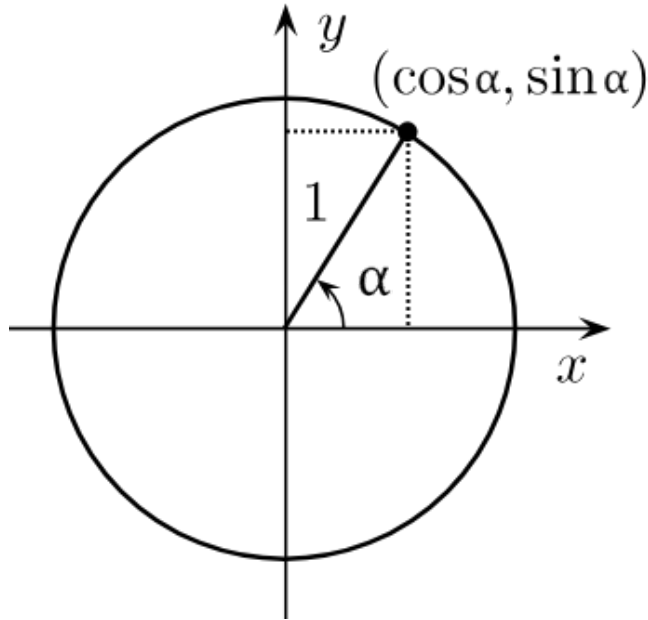
Angolo in gradi	Angolo in radianti
0°	0
30°	$\pi/6$
45°	$\pi/4$
60°	$\pi/3$
90°	$\pi/2$
180°	π
270°	$3\pi/2$
360°	2π

PI



Trigonometria

SENO E COSENO

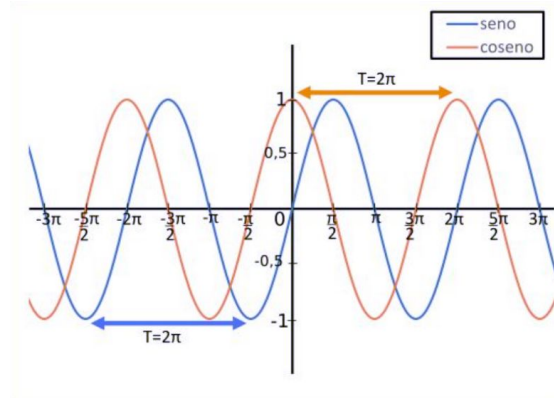


Circonferenza raggio 1

- Cos: proiezione su asse X
- Sin: proiezione su asse Y

Utilizzi comuni

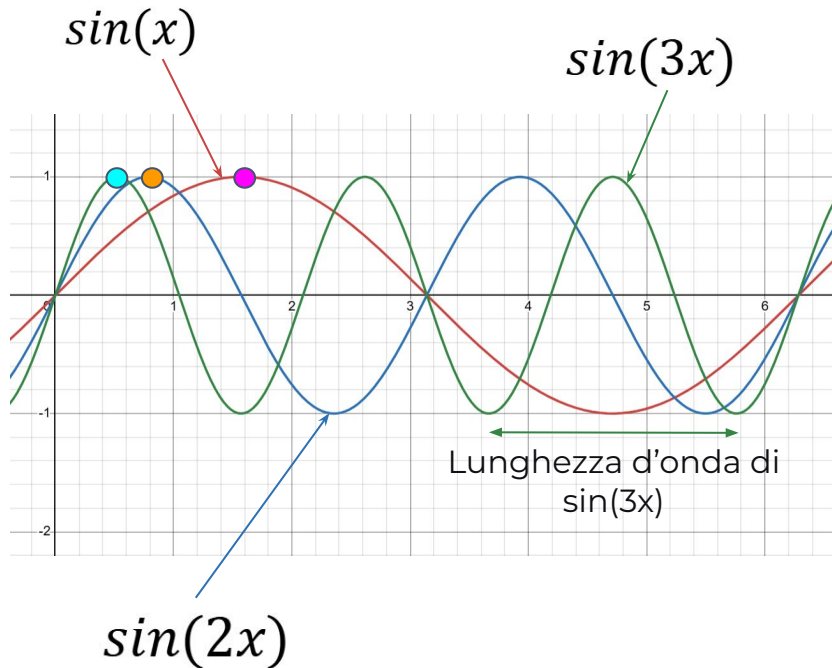
- Calcoli di angoli
- Matrici di rotazione
- Coordinate polari





Trigonometria

SENO E COSENO



Moltiplicare il valore di input di una funzione periodica cambia la **frequenza**

- $\sin(x)$ raggiunge 1 quando x vale $\pi/2$
- $\sin(2x)$ raggiunge 1 quando x vale $\pi/4$
- $\sin(3x)$ raggiunge 1 quando x vale $\pi/6$

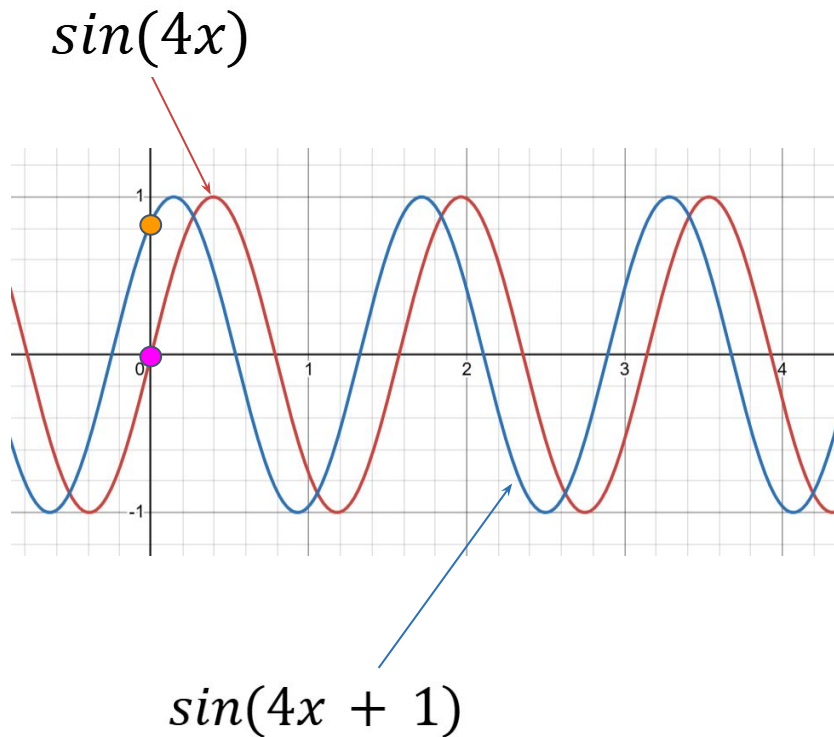
La **lunghezza d'onda** è la **distanza** (sull'asse x) tra due **massimi/minimi** di una funzione periodica. Cambia in modo **inversamente proporzionale** rispetto alla frequenza.

Lunghezza d'onda = $1 / \text{frequenza}$



Trigonometria

SENO E COSENO



Aggiungere un valore al valore di input di una funzione periodica cambia la **fase**

- $\sin(4x)$ con $x = 0$ vale $\sin(0) = 0$
- $\sin(4x+1)$ con $x = 0$ vale $\sin(1) = 0.84147$

Aggiungere un valore **positivo** sposta il grafico della funzione periodica verso **sinistra**

Aggiungere un valore **negativo** sposta il grafico della funzione periodica verso **destra**

Sommare/sottrarre un valore all'input della funzione periodica **non cambia la frequenza**

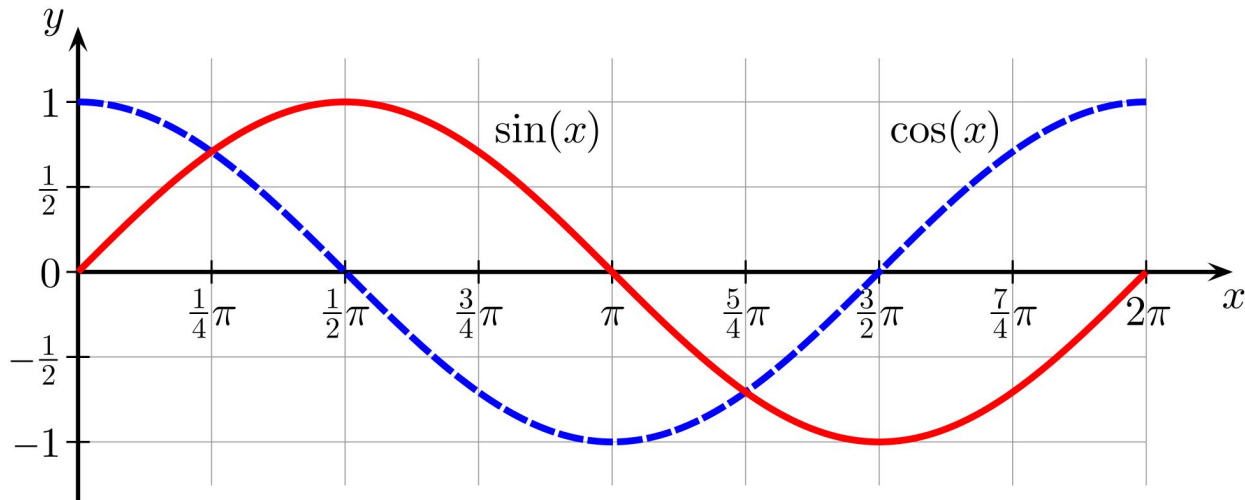


Trigonometria

ESERCIZI

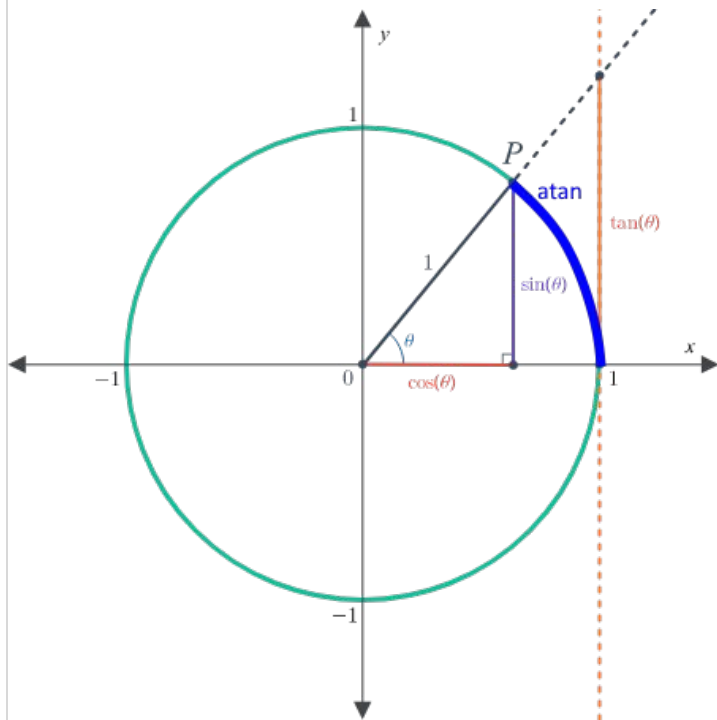
Consideriamo il movimento **in una direzione arbitraria**

1. **Movimento ondulatorio:** muovere un oggetto con moto ondulatorio
2. **Movimento ondulatorio variabile:** muovere un oggetto con moto ondulatorio, con velocità/frequenza tunabile
3. **Effetto onda:** muovere più oggetti con moto ondulatorio, con velocità/frequenza e offset/fase tunabili, creando un effetto onda





Trigonometria



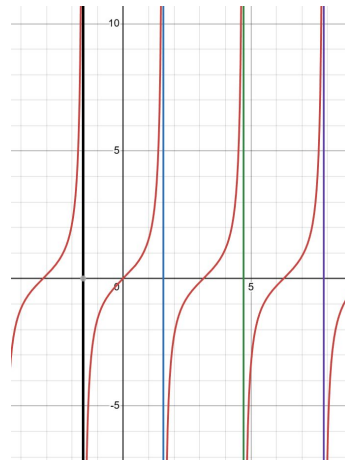
Prima relazione fondamentale

$$\cos^2 x + \sin^2 x = 1$$

Seconda relazione fondamentale

$$\tan x = \frac{\sin x}{\cos x}$$

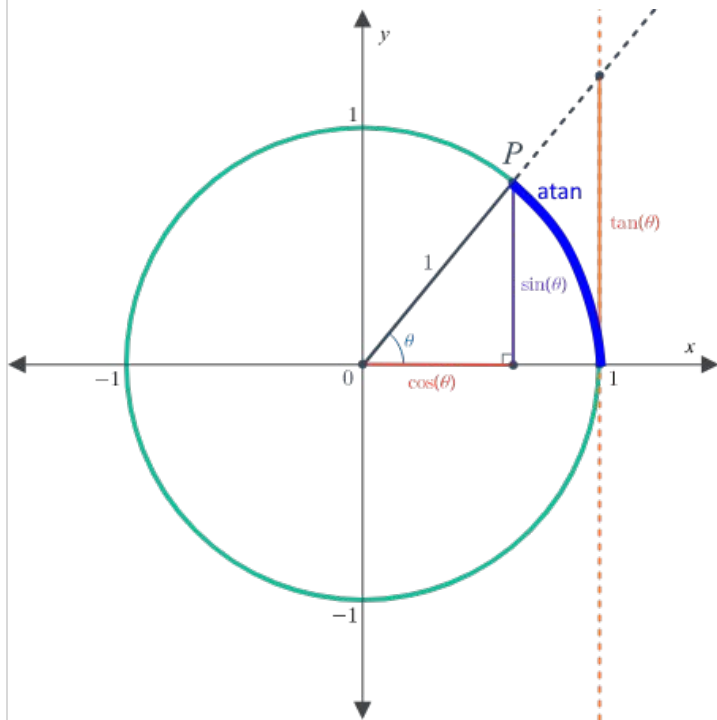
La tangente a $\pi/2$ e multipli
va **all'infinito**





Trigonometria

COORDINATE POLARI



Coordinate polari circolari

$$x = r \cdot \cos(\alpha)$$

$$y = r \cdot \sin(\alpha)$$

r = distanza dal centro

α = direzione angolare rispetto all'asse X

Cerchio con origine in (x_0, y_0)

$$\begin{cases} x = x_0 + r \cdot \cos(\alpha) \\ y = y_0 + r \cdot \sin(\alpha) \end{cases}$$

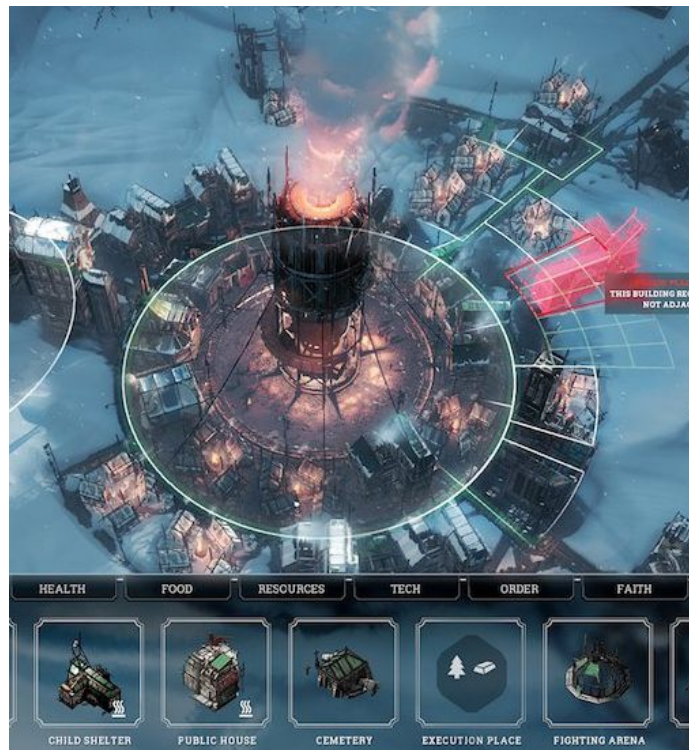


Trigonometria

COORDINATE POLARI - ESERCIZI

Movimento radiale

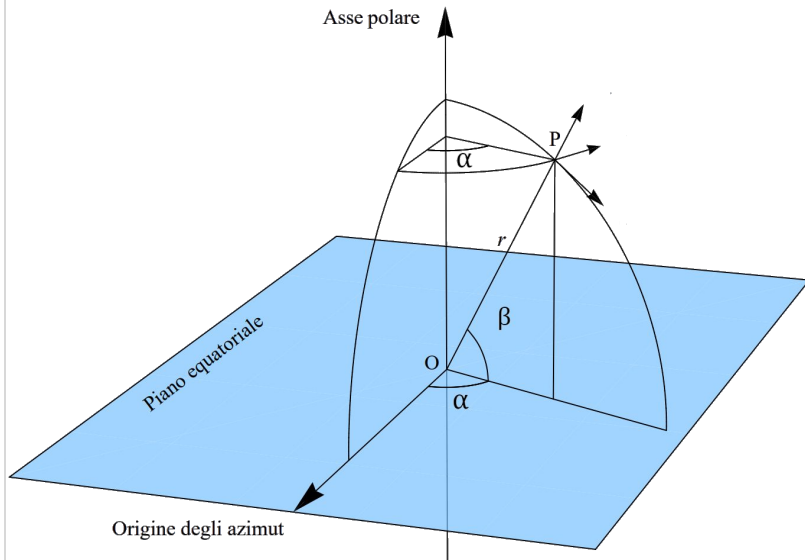
1. Muovere un actor su una **griglia radiale** con seguenti input
 - a. Avanti/indietro per cambiare distanza dal **centro**
 - b. Sinistra/destra per ruotare rispetto all'**asse Z**
2. Rendere il movimento radiale a **velocità costante** (ovvero velocità radiale che diminuisce all'aumentare della distanza)
3. **Orientare** l'actor **verso il centro** in modo che guardi sempre quel punto a qualsiasi distanza e angolo





Trigonometria

COORDINATE POLARI



Coordinate polari sferiche

$$x = r \cdot \cos(\alpha) \cdot \cos(\beta)$$

$$y = r \cdot \sin(\alpha) \cdot \cos(\beta)$$

$$z = r \cdot \sin(\beta)$$

r = distanza dal centro

α = rotazione attorno l'asse polare sul piano equatoriale

β = rotazione dell'asse polare che da origine alle proiezioni sul piano equatoriale

Sfera con origine in (x_0, y_0, z_0)

$$\begin{cases} x = x_0 + r \cdot \cos(\alpha) \cdot \cos(\beta) \\ y = y_0 + r \cdot \sin(\alpha) \cdot \cos(\beta) \\ z = z_0 + r \cdot \sin(\beta) \end{cases}$$

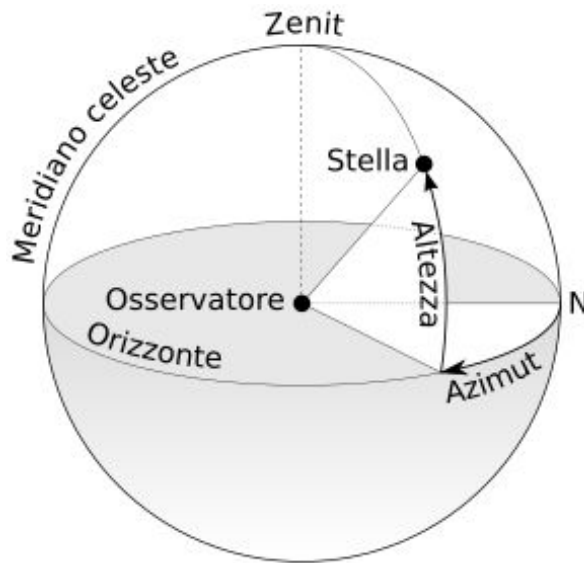


Trigonometria

COORDINATE POLARI - ESERCIZI

Movimento sferico

1. Muovere un actor in uno spazio sferico con seguenti input
 - a. Avanti/indietro per cambiare distanza dal **centro**
 - b. Sinistra/destra per ruotare rispetto all'**asse Z**
 - c. Q/E per ruotare al piano equatoriale
2. Rendere il movimento radiale a **velocità costante** (ovvero velocità radiale che diminuisce all'aumentare della distanza)
3. **Orientare** l'actor **verso il centro** in modo che guardi sempre quel punto a qualsiasi distanza e angolo





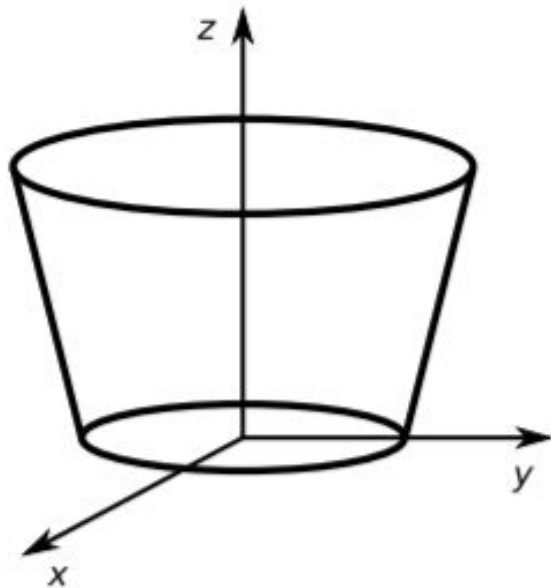
Trigonometria

COORDINATE POLARI - ESERCIZI

Movimento a spirale

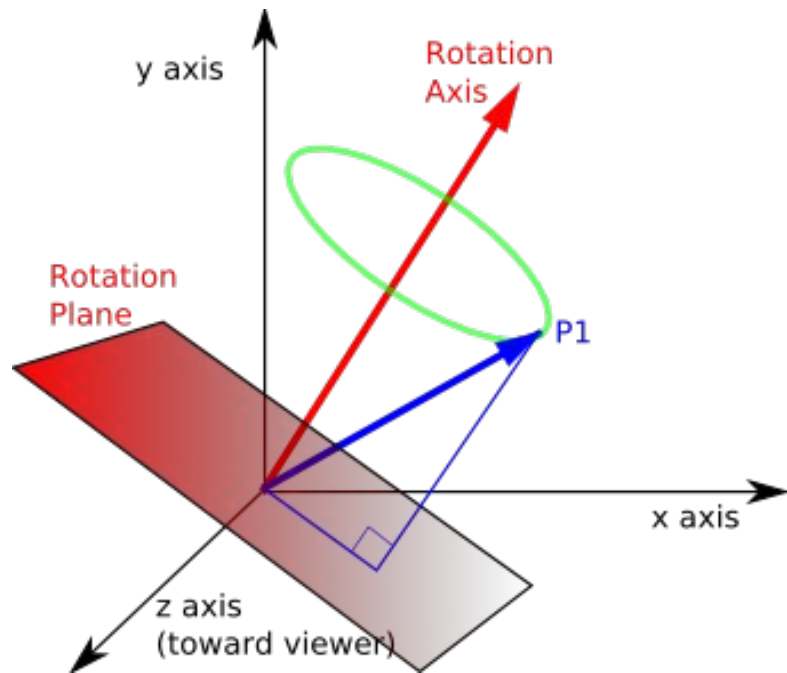
Muovere un actor nello spazio 3D con le coordinate polari **scegliendo l'asse** della spirale a scelta tra X, Y e Z

1. L'intensità del movimento sull'asse deve essere tunabile
2. Il raggio **minimo** e il raggio **massimo** della spirale devono essere tunabili ed essere interpolati per avere un effetto *sezione di cono* (immagine a destra)
3. Utilizzando l'interpolazione lineare, rendere il movimento a **tempo tunabile e prestabilito**





Quaternioni



Un quaternione **rappresenta una rotazione nello spazio 3D**. I quaternioni sono basati su numeri complessi e non sono semplici da capire in modo intuitivo.

Un quaternione si può scomporre in

- **asse di rotazione**(x, y, z)
- **intensità di rotazione**(w)

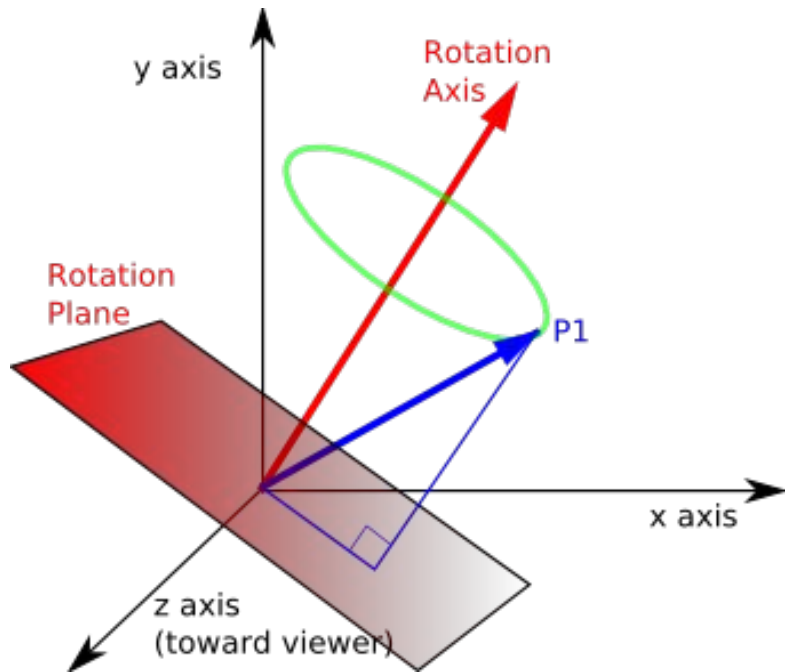
In ambito videoludico i quaternioni vengono usati per:

- definire l'**orientamento** di un oggetto nello spazio 3D
- **interpolare** due rotazioni 3D
- evitare il **gimbal lock**



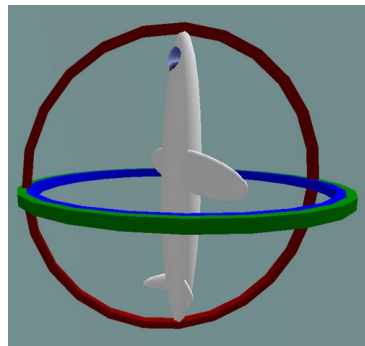
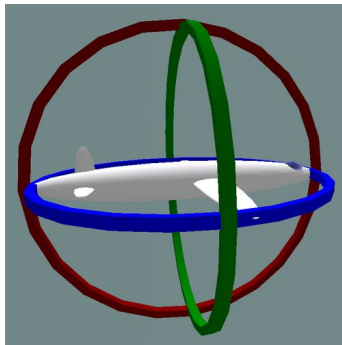
Quaternioni

GIMBAL LOCK



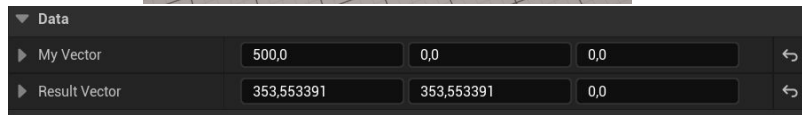
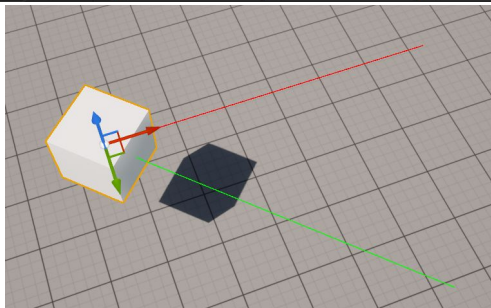
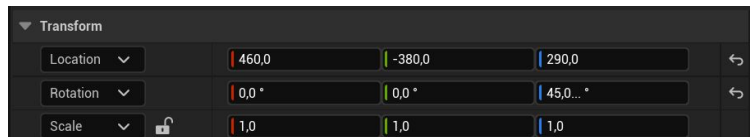
Il **gimbal lock (blocco cardanico)** è una perdita di un grado di libertà dovuto ad un allineamento di più assi

Per **evitarlo** bisogna applicare rotazioni mediante quaternioni (in Unreal anche con i **Rotators**) e non mediante la semplice somma degli angoli





Quaternioni



Moltiplicare un quaternione per un vettore da come risultato un **vettore ruotato**.

Il vettore ruotato è **allineato** con il **quaternione** come il vettore iniziale è allineato con l'origine dello spazio 3D

$$V_a = Q_a * V$$

In Unreal si usa la funzione **RotateVector** nei blueprints, mentre a codice si utilizza l'operatore *****. Moltiplicare due quaternioni tra di loro equivale ad avere una **rotazione combinata**

$$Q_c = Q_a * Q_b$$

Non è commutativa $Q_a * Q_b \neq Q_b * Q_a$

Nell'esempio qui sopra la **linea rossa** rappresenta il vettore di partenza **My Vector** mentre **Result Vector** è rappresentato dalla **linea verde**. Il cubo è girato di 45° sull'asse Z e quindi moltiplicare la sua rotazione per il **My Vector** produce un vettore ruotato proprio di 45° sull'asse Z. Guardando **dal punto di vista del cubo** il vettore **Result Vector** ha una posizione **relativa** di 500,0,0 (ovvero **My Vector**)



Quaternioni

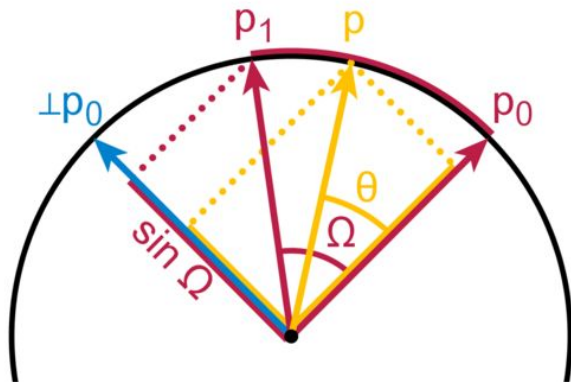
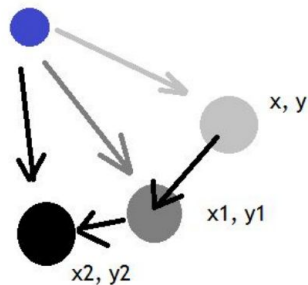
ESERCIZI

Inseguimento angolare

1. Progetto **FPS** template
2. Aggiungere **actor *nemico* con un verso** - (esempio: cubo)

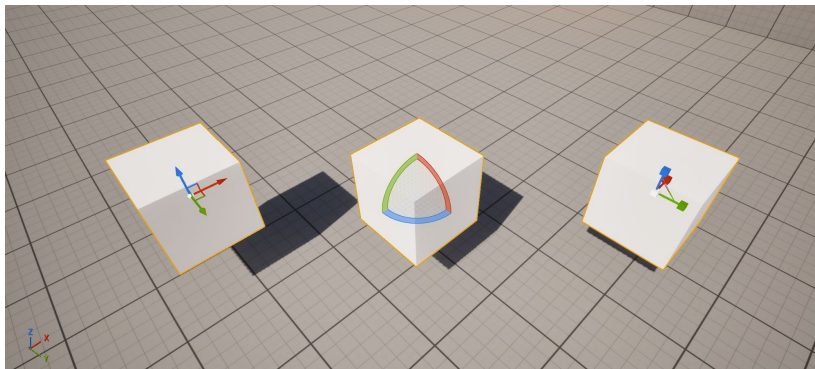
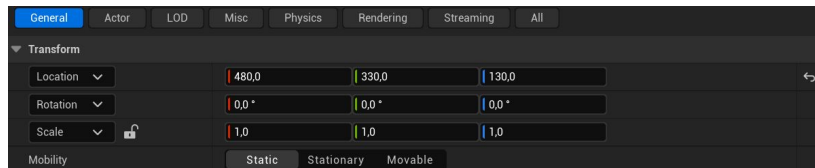
Esercizi

1. **Rotazione verso il Player:** usando *Find Look at Rotation* far seguire con lo sguardo il *player pawn* dall'oggetto *nemico*
2. **Rotazione verso il Player interpolata:** usando i quaternioni e la funzione *FQuat::Slerp* far seguire con lo sguardo il *Player Pawn* dall'actor *Nemico*, rendendo tunabile la **reattività** dell'azione
3. **Inseguimento interpolato:** l'actor *nemico* deve inseguire il *Player Pawn*, ruotando l'inseguitore mentre è in fase di inseguimento e muovendosi verso la propria direzione *forward*





Transform



Una transform definisce **l'origine di uno spazio specifico**

Permette di definire

- Posizione(traslazione)
- Rotazione(orientamento)
- Scala

Transform identità (World Space)

Posizione: 0, 0, 0

Rotazione: 0, 0, 0

Scala: 1, 1, 1

La rotazione è rappresentata mediante **3 angoli, detti angoli di Eulero**, ognuno rispetto l'asse corrispondente



Transform

Transform Direction

- *Input:* Direzione (local space)
- *Output:* Direzione (world space)
- Non cambia lunghezza

Direzione relativa alla transform → direzione world space

Transform Location

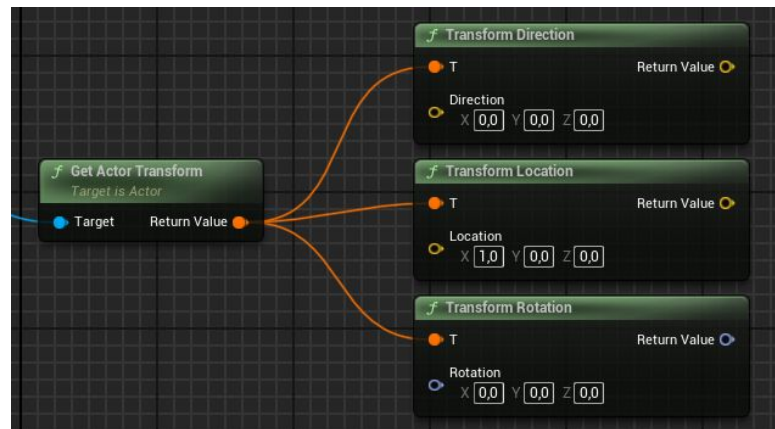
- *Input:* Posizione (local space)
- *Output:* Posizione (world space)
- Tiene conto della scala

Posizione relativa alla transform → posizione world space

Transform Rotation

- *Input:* Rotazione (local space)
- *Output:* Rotazione (world space)

Rotazione relativa alla transform → rotazione world space

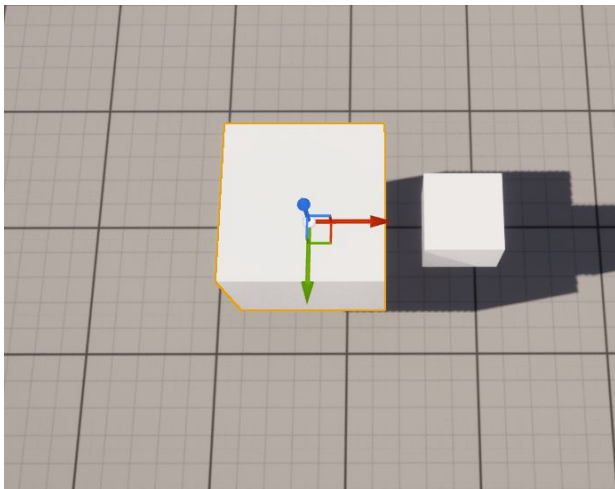
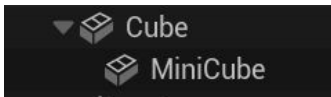




Transform

Esempio: Transform Position

L'actor *Cube* ha un *child* actor di nome *MiniCube* che è messo alla posizione relativa (100, 0, 0)



MiniCube
MiniCube (Instance)
StaticMeshComponent (StaticMeshComponent0)

Search

General Actor LOD Misc Physics Rendering Streaming All

Transform
Location 100,0 0,0 0,0
Rotation 0,0 ° 0,0 ° 0,0 °
Scale 0,5 0,5 0,5

Transform
Absolute Location 580,0 330,0 130,0
Rotation 0,0 ° 0,0 ° 0,0 °
Scale 0,5 0,5 0,5

Cube
Cube (Instance)
StaticMeshComponent (StaticMeshComponent0)

Search

General Actor LOD Misc Physics Rendering Streaming All

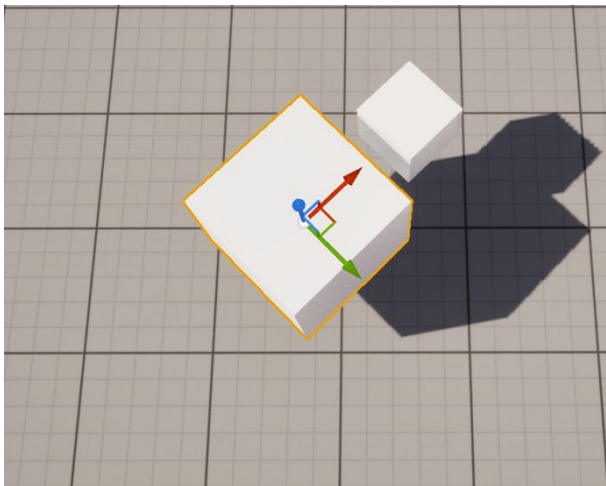
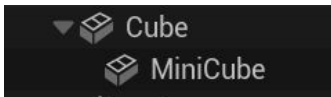
Transform
Location 480,0 330,0 130,0
Rotation 0,0 ° 0,0 ° 0,0 °
Scale 1,0 1,0 1,0



Transform

Esempio: Transform Position

Ruotando *Cube* la posizione world (Absolute location) di *MiniCube* cambia



MiniCube

MiniCube (Instance)

StaticMeshComponent (StaticMeshComponent0)

Search

General Actor LOD Misc Physics Rendering Streaming All

Transform

Location	100,0	0,0	0,0
Rotation	0,0 °	0,0 °	0,0 °
Scale	0,5	0,5	0,5

Cube

Cube (Instance)

StaticMeshComponent (StaticMeshComponent0)

Search

General Actor LOD Misc Physics Rendering Streaming All

Transform

Absolute Location	550,710678	259,289322	130,0
Rotation	0,0 °	0,0 °	0,0 °
Scale	0,5	0,5	0,5



Transform

Inverse Transform Direction

- *Input:* Direzione(world space)
- *Output:* Direzione (local space)
- Non cambia lunghezza

Direzione world space → direzione relativa alla transform

Inverse Transform Location

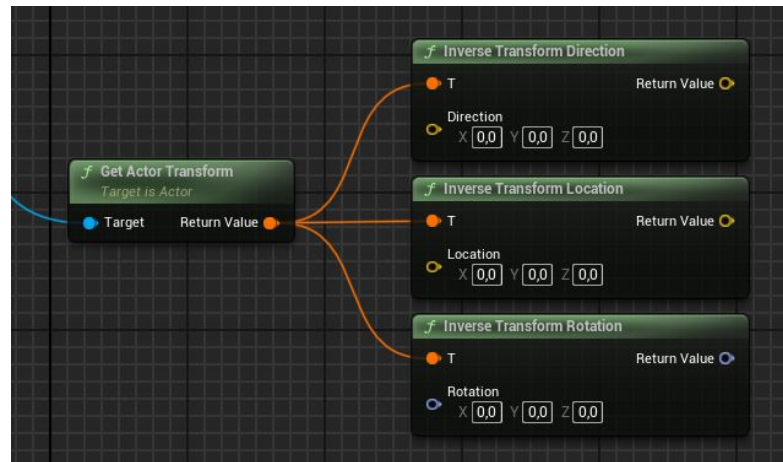
- *Input:* Posizione (world space)
- *Output:* Posizione (local space)
- Tiene conto della scala

Posizione world space → posizione relativa alla transform

Inverse Transform Rotation

- *Input:* Rotazione (world space)
- *Output:* Rotazione (local space)

Rotazione world space → rotazione relativa alla transform

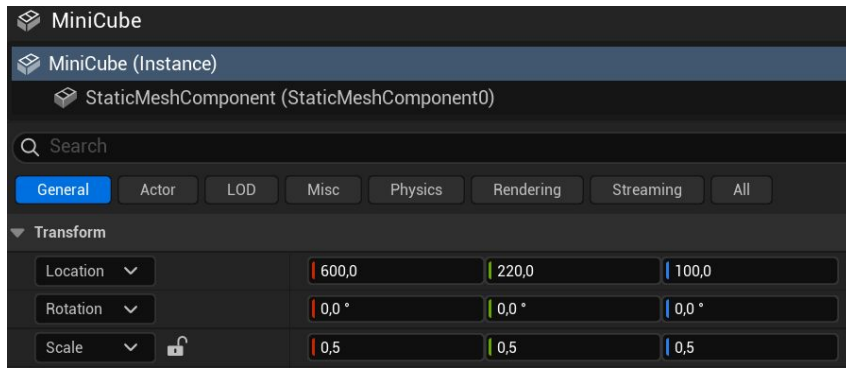
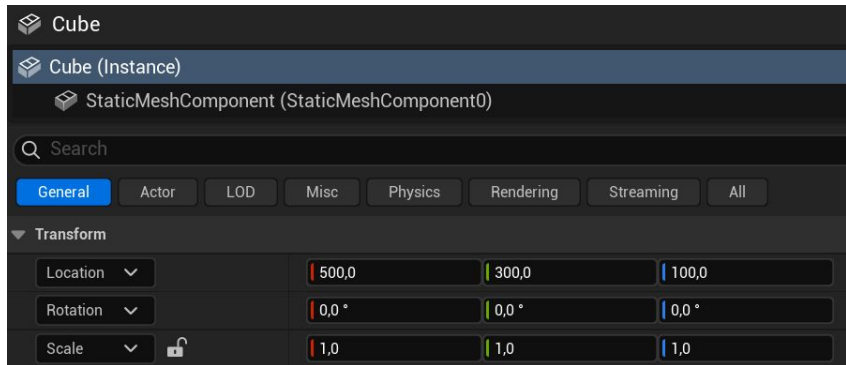
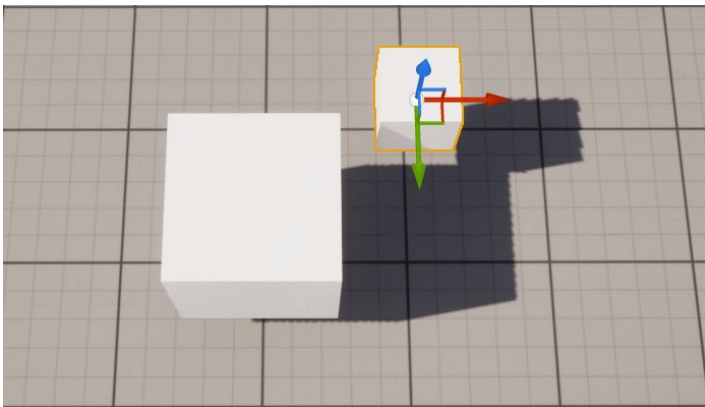
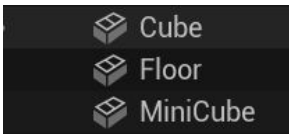




Transform

Esempio: Inverse Transform Position

Gli actor *Cube* e *MiniCube* sono posizionati nel mondo

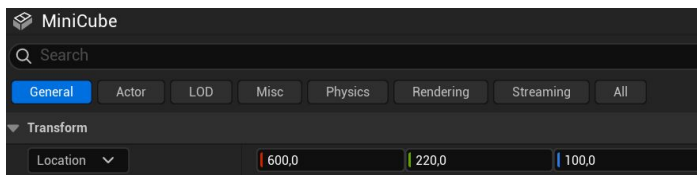
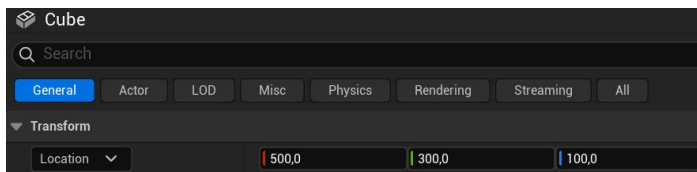
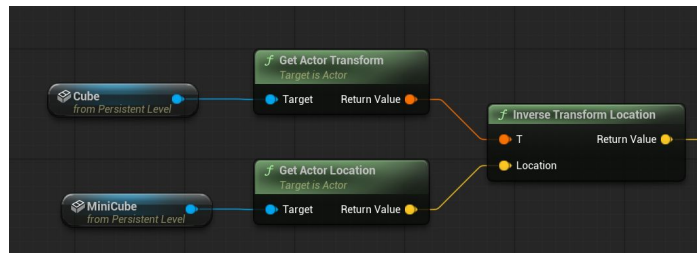
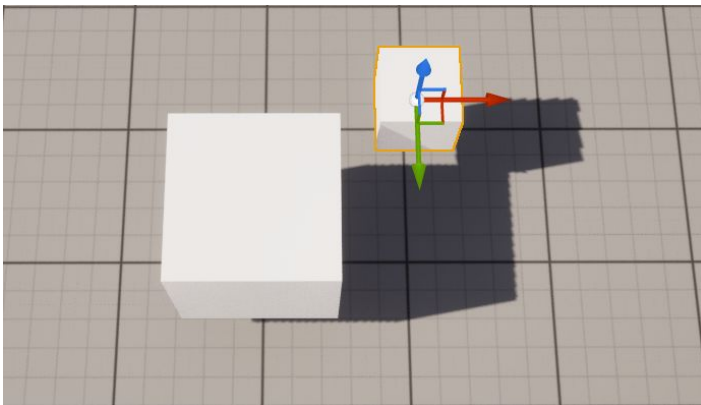
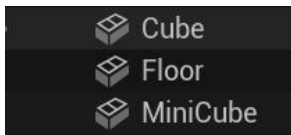




Transform

Esempio: Inverse Transform Position

Facendo Inverse Transform Position della location di *MiniCube* rispetto la trasform di *Cube* ottengo la posizione relativa



X=100.000 Y=-80.000 Z=0.000

'DisableAllScreenMessages' to suppress

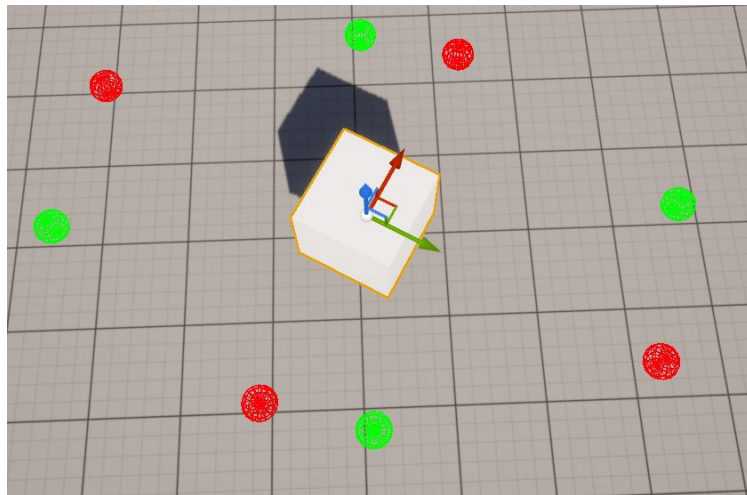
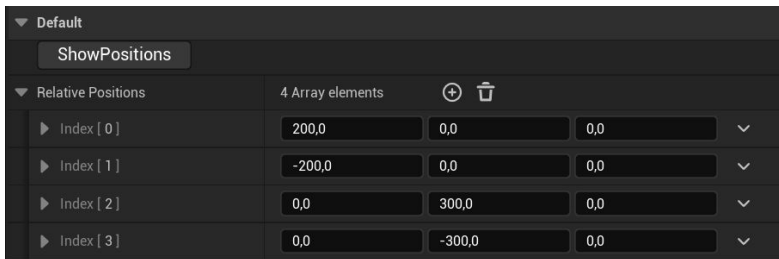


Transform

ESERCIZI

Debug drawing

1. Creare il blueprint di un actor con una variable - **array di punti** (vettori).
2. Fare una funzione (*Call in Editor*) per vedere in modalità di debug i punti utilizzati come **semplice traslazione** dal **centro dell'actor**
3. Fare una funzione per vedere in modalità di debug la posizione nel **mondo** dei punti utilizzati se il loro **valore** fosse **la posizione relativa rispetto all'actor**





Distanze

La distanza viene spesso utilizzata per sistemi di ottimizzazione

Esempio:

Gestione tick Actors - Distanza dalla camera

1. Distanza di disattivazione
2. Distanza di attivazione (di solito inferiore a quella di disattivazione)

I **controlli di distanza** vengono ottimizzati controllando il **quadrato** delle distanze perché si **evita** di calcolare **radici quadrate**

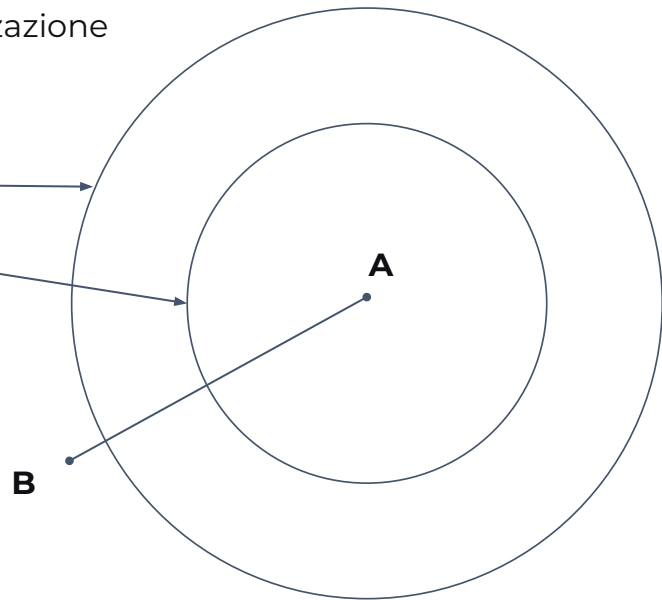
CPU Time Sqrt > CPU Time Moltiplicazione

Invece di

$Distance(A,B) \leftrightarrow Distanza\ da\ controllare$

è più ottimizzato

$DistanceSquared(A,B) \leftrightarrow Distanza\ da\ controllare * Distanza\ da\ controllare$





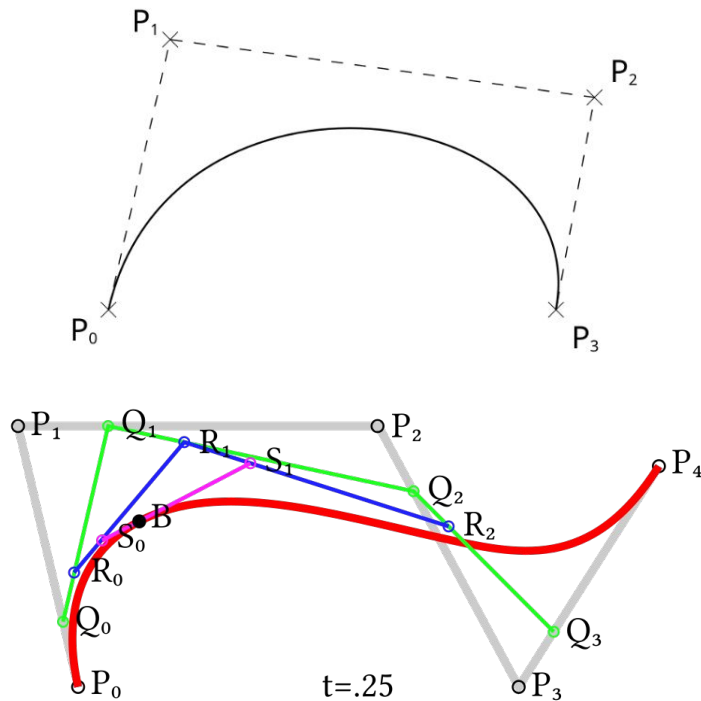
Curve di Bézier

Le **curve di Bézier** sono curve **parametriche** generate da un numero finito di **punti** e dalla **variazione** del **parametro di interpolazione** da 0 a 1

Utilizzate per

- Percorsi curvi
- Superfici curve
- Carrozzerie automobili

Si possono definire l'**insieme dei punti di interpolazione** tra più punti





Curve di Bézier

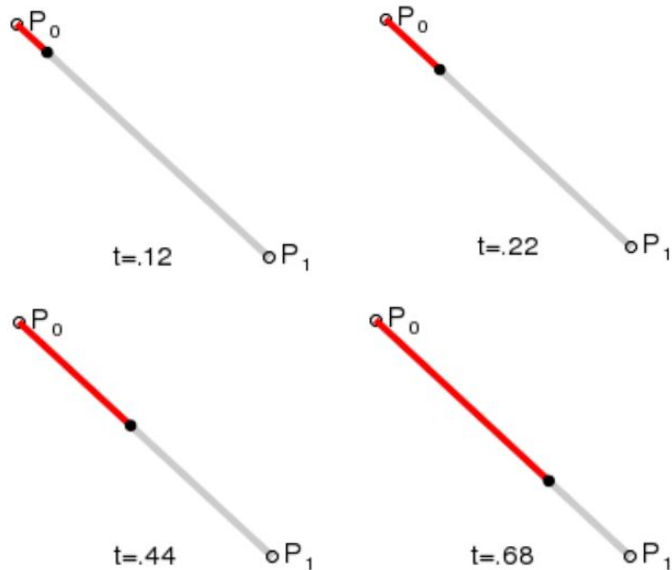
Curve Lineari

Curva tra **due punti** che descrive il tragitto dal primo al secondo punto

Si ottiene facendo l'interpolazione tra i due punti e variando il parametro di interpolazione

$$B(t) = (1 - t) \cdot P_0 + t \cdot P_1$$

Essendo di primo grado l'insieme dei punti è un **segmento** che va da P_0 a P_1





Curve di Bézier

Curve Quadratiche

Curva tra **tre punti** che descrive il tragitto dal primo al terzo punto, passando vicino al secondo

Si ottiene facendo **l'interpolazione** da i **due punti ottenuti** da

1. L'interpolazione tra **1°** punto e **2°** punto
2. L'interpolazione tra **2°** punto e **3°** punto

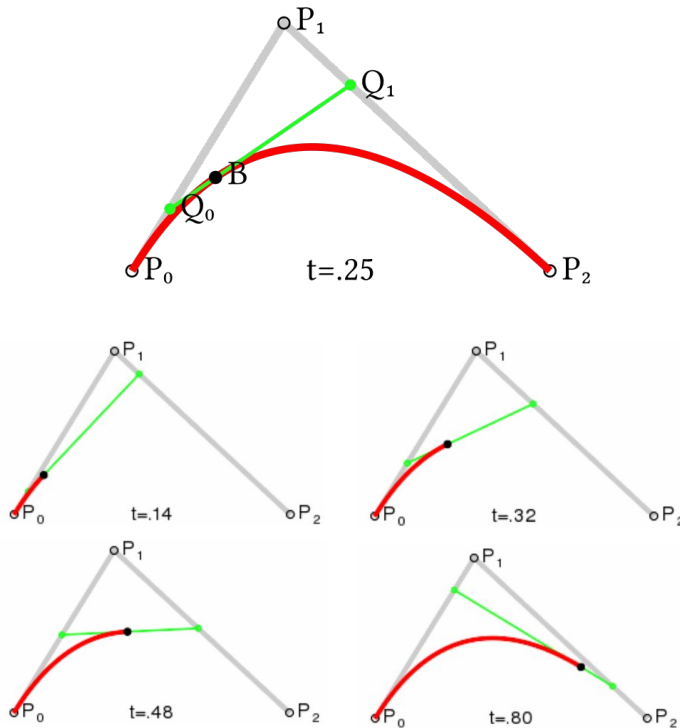
$$Q_0(t) = (1 - t) \cdot P_0 + t \cdot P_1$$

$$Q_1(t) = (1 - t) \cdot P_1 + t \cdot P_2$$

Interpolando questi punti si ottiene

$$B(t) = (1 - t) \cdot Q_0 + t \cdot Q_1$$

Essendo di **secondo** grado l'insieme dei punti è una **parabola** che va da P_0 a P_2





Curve di Bézier

Curve Quadratiche

Sostituiamo Q_0 e Q_1 nella formula di interpolazione

$$B(t) = (1 - t) \cdot Q_0 + t \cdot Q_1$$

$$B(t) = (1 - t) \cdot ((1 - t) \cdot P_0 + t \cdot P_1) + t \cdot ((1 - t) \cdot P_1 + t \cdot P_2)$$

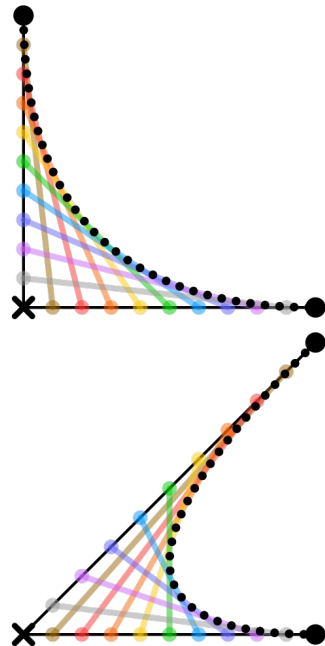
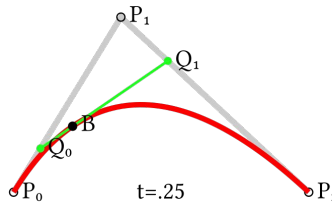
RisolviAMO raggruppando per i punti che generano la curva

$$B(t) = (1 - t)^2 \cdot P_0 + t \cdot (1 - t) \cdot P_1 + t \cdot (1 - t) \cdot P_1 + t^2 \cdot P_2$$

$$B(t) = (1 - t)^2 P_0 + 2t(1 - t)P_1 + t^2 P_2$$

Polinomio di secondo grado

(il segno di moltiplicazione è implicito tra due termini vicini tra loro)





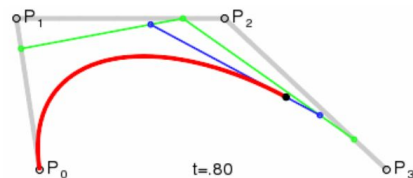
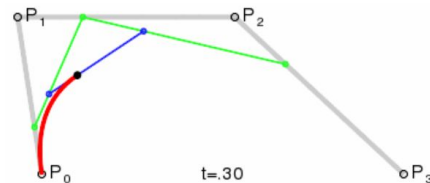
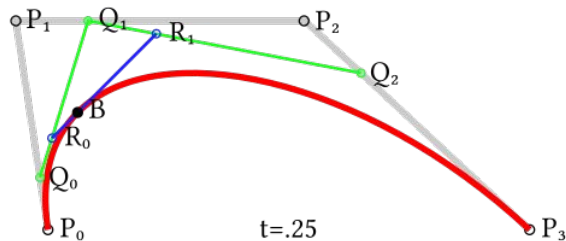
Curve di Bézier

Curve Cubiche

Curva tra **quattro punti** che descrive il tragitto dal primo al quarto punto, passando vicino al secondo e al terzo R_0

Si ottiene facendo **l'interpolazione** da i **due punti** **ottenuti** da

1. L'interpolazione R_0 tra **1° punto**, **2° punto** e il **3° punto** che si ottiene **interpolando i due punti** ottenuti da
 - a. L'interpolazione Q_0 tra **1° punto** e **2° punto**
 - b. L'interpolazione Q_1 tra **2° punto** e **3° punto**
2. L'interpolazione R_1 tra **2° punto**, **3° punto** e il **4° punto** che si ottiene **interpolando i due punti** ottenuti da
 - a. L'interpolazione Q_1 tra **2° punto** e **3° punto**
 - b. L'interpolazione Q_2 tra **3° punto** e **4° punto**





Curve di Bézier

Curve Cubiche

Sostituiamo R_0 e R_1 nella formula di interpolazione

$$B(t) = (1 - t) \cdot R_0 + t \cdot R_1$$

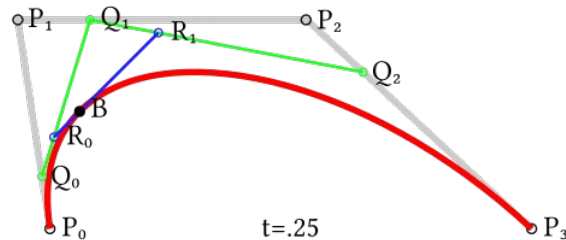
$$B(t) = (1 - t) \cdot ((1 - t) \cdot Q_0 + t \cdot Q_1) + t \cdot ((1 - t) \cdot Q_1 + t \cdot Q_2)$$

Sostituendo anche Q_0, Q_1 e Q_2 , e raggruppando per i punti della iniziali si ottiene

$$B(t) = (1 - t)^3 P_0 + 3t(1 - t)^2 P_1 + 3t^2(1 - t) P_2 + t^3 P_3$$

Polinomio di terzo grado

(il segno di moltiplicazione è implicito tra due termini vicini tra loro)





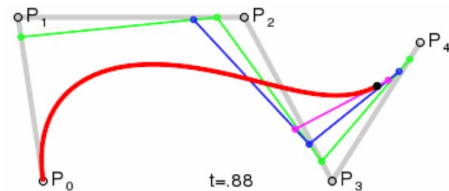
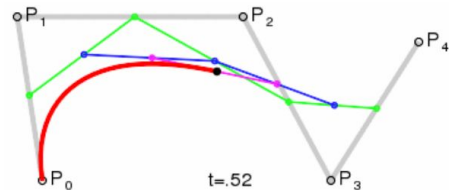
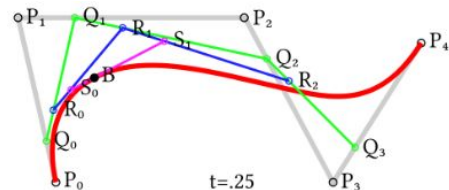
Curve di Bézier

Curve con N punti

Curva tra **N punti** che descrive il tragitto dal primo al N-esimo punto, passando vicino a tutti i punti intermedi

Si ottiene facendo **l'interpolazione tra due punti** modo **ricorsivo**

1. L'interpolazione dal **1 al N-1** punto ottenuta da
 - a. Interpolazione dal **1 al N-2** punto ottenuta da
 - i. Interpolazione dal **1 al N-3** punto ottenuta da
 1. ...
 - a. **Interpolazione dal punto 1 al punto 2**
 - b. **Interpolazione dal punto 2 al punto 3**
 - ii. Interpolazione dal **2 al N-2** punto ottenuta da
 - b. Interpolazione dal **2 al N-1** punto ottenuta da
 - i. ...
 2. L'interpolazione dal **2 al N** punto ottenuta da
 - a. Interpolazione dal **2 al N-1** punto ottenuta da
 - i. ...
 - b. Interpolazione dal **3 al N** punto ottenuta da
 - i. ...





Curve di Bézier

Curve con N punti - primo metodo

Per trovare il valore al parametro t bisogna fare

$$B(t) = (1 - t)B_{0...n-1}(t) + tB_{1...n}(t)$$

che si traduce nella somma di

$$B_{0...n-1} = (1 - t)B_{0...n-2}(t) + tB_{1...n-1}(t)$$

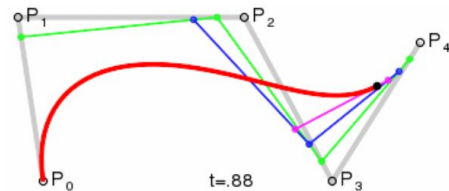
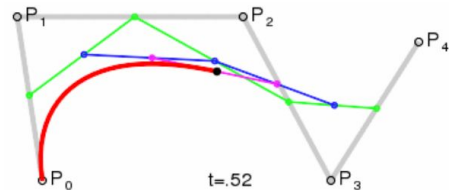
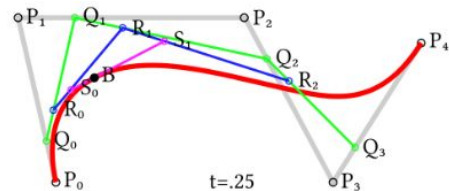
$$B_{1...n} = (1 - t)B_{1...n-1}(t) + tB_{2...n}(t)$$

Per risolvere questa curva si utilizza una funzione **ricorsiva** il cui **caso base** corrisponde con la curva di **Bézier lineare**

```
return (1 - t) * Array[0] + t * Array[1]
```

Il valore di ritorno del caso **n-esimo** della funzione è

```
return (1 - t) * Bezier(ArrayRange(0, ArrayNum - 2)) +  
      t * Bezier (ArrayRange(1, ArrayNum - 1))
```





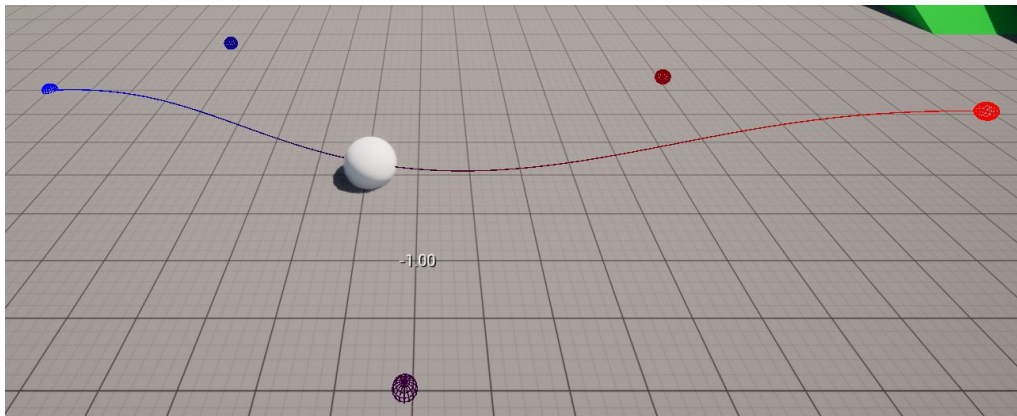
Curve di Bézier

ESERCIZI

Percorso a spline - primo metodo

Creare un percorso a spline usando **l'algoritmo ricorsivo** per le curve di Bézier con le seguenti specifiche

1. Il **numero** di punti da cui generare la curva deve essere tunabile e non fisso
2. I **punti** di generazione devono essere **visibili** in debug con funzione *Call In Editor*
3. Implementare un **oggetto** che si **sposta** sulla **spline** generata in un tempo prestabilito e tunabile
4. **Visualizzare la spline** in debug con funzione *Call In Editor*





Curve di Bézier

Curve con N punti - secondo metodo

Le curve di Bézier possono essere risolte anche **senza ricorsione**

Lineare

$$B(t) = (1 - t) \cdot P_0 + t \cdot P_1$$

Quadratico

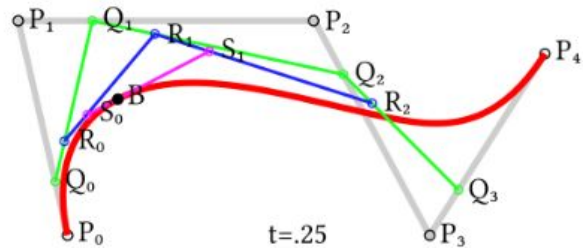
$$B(t) = (1 - t)^2 P_0 + 2t(1 - t)P_1 + t^2 P_2$$

Cubico

$$B(t) = (1 - t)^3 P_0 + 3t(1 - t)^2 P_1 + 3t^2(1 - t)P_2 + t^3 P_3$$

Si può generalizzare la formula nel seguente modo

$$B(t) = \sum_{i=0}^n \binom{n}{i} t^i (1 - t)^{n-i} P_i \quad \text{con} \quad \binom{n}{i} = \frac{n!}{(n-i)!i!} = \frac{1 \cdot 2 \cdot \dots \cdot n}{[1 \cdot 2 \cdot \dots \cdot (n-1)](1 \cdot 2 \cdot \dots \cdot i)}$$



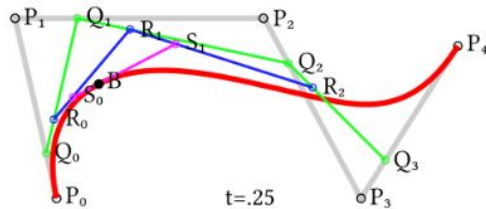


Curve di Bézier

Curve con N punti - secondo metodo

$$B(t) = \sum_{i=0}^n \binom{n}{i} t^i (1-t)^{n-i} P_i$$

significa quindi sommare ogni punto da P_0 a P_n calcolando i coefficienti in base a t, i ed n . Questa notazione si chiama **polinomi di base di Bernstein**



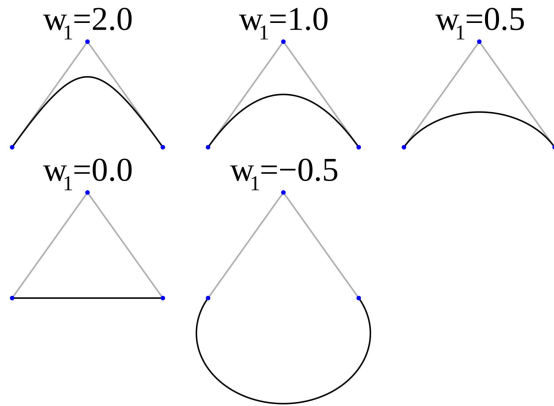
Curve razionali/pesate

Per cambiare l'andamento della curva si possono utilizzare dei **pesi** per ciascun punto con la seguente formula

$$B(t) = \frac{\sum_{i=0}^n \binom{n}{i} t^i (1-t)^{n-i} P_i w_i}{\sum_{i=0}^n \binom{n}{i} t^i (1-t)^{n-i} w_i}$$

dove w_i è il peso dell' i -esimo punto di generazione

Notare come il **peso** di ogni punto **cambia la forma** della curva, in particolare per valori negativi o nulli





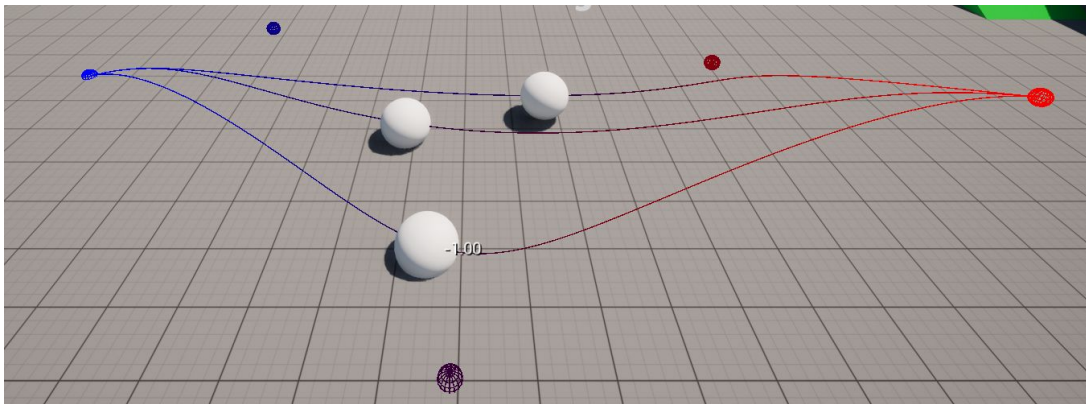
Curve di Bézier

ESERCIZI

Percorso a spline - secondo metodo

Creare un percorso a spline usando **l'algoritmo con i polinomi nella base di Bernstein** per le curve di Bézier con le seguenti specifiche

1. Il **numero** di punti da cui generare la curva deve essere tunabile e non fisso
2. Il **peso** di ogni punto deve essere tunabile insieme al punto stesso
3. I **punti** di generazione devono essere **visibili** in debug con funzione *Call In Editor*
4. Implementare un **oggetto** che si **sposta** sulla **spline** generata in un tempo prestabilito e tunabile
5. **Visualizzare la spline** in debug con funzione *Call In Editor*





Simulazione fisica

Un movimento può essere simulato seguendo la sua **cinematica**

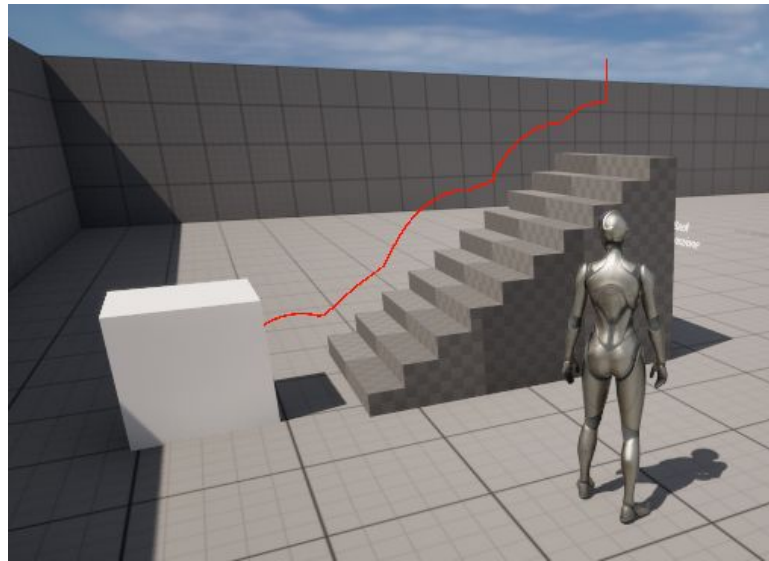
Le **informazioni** necessarie per poter simulare un movimento sono

1. **Posizione**
2. **Rotazione**
3. **Istante di tempo**

Posizione e rotazione vengono **campionate** in determinati *istanti di tempo*, e vengono poi riprodotte **interpolando** le posizioni e rotazioni non registrate in tutti gli altri istanti di tempo/frames

Interpolazioni

- Posizione: `FVector::Lerp()`
- Rotazione: `FQuat::Slerp()`



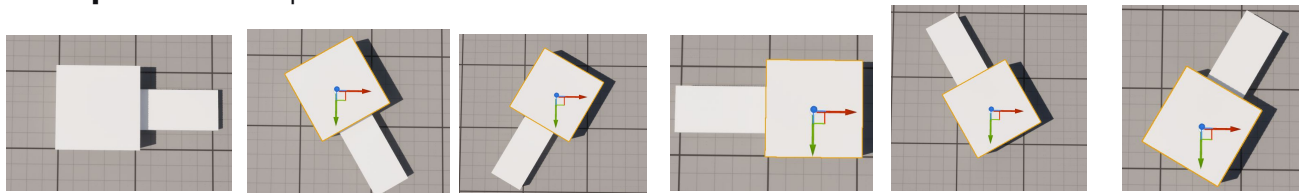


Simulazione fisica

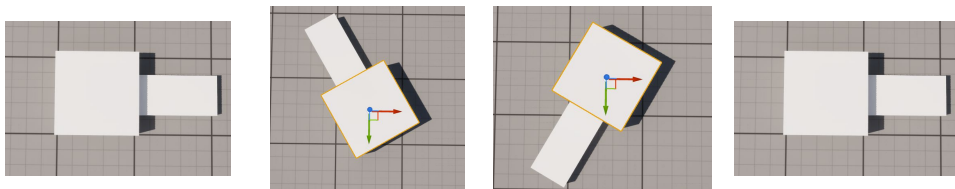
ALIASING

Attenzione all'**aliasing**! Quando il campionamento non è **sufficientemente** fitto, si potrebbe avere aliasing, causando un movimento riprodotto diverso dall'originale

Esempio: il cubo qui sotto sta ruotando in senso **orario**



Se gli *istanti di tempo/frames* fossero **troppo distanti** tra loro come qui sotto



l'interpolazione porterebbe a far ruotare il cubo in senso **antiorario**

Per sapere quanto **frequentemente** campionare i dati si può usare il seguente **teorema**
https://it.wikipedia.org/wiki/Teorema_del_campionamento_di_Nyquist-Shannon



Simulazione fisica

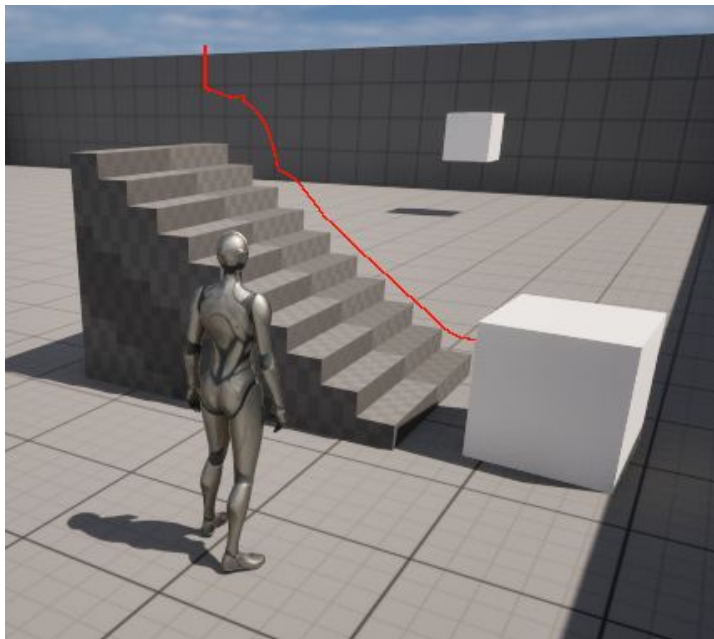
ESERCIZI

Registrazione e riproduzione

Creare un sistema per registrare e riprodurre il movimento di un oggetto

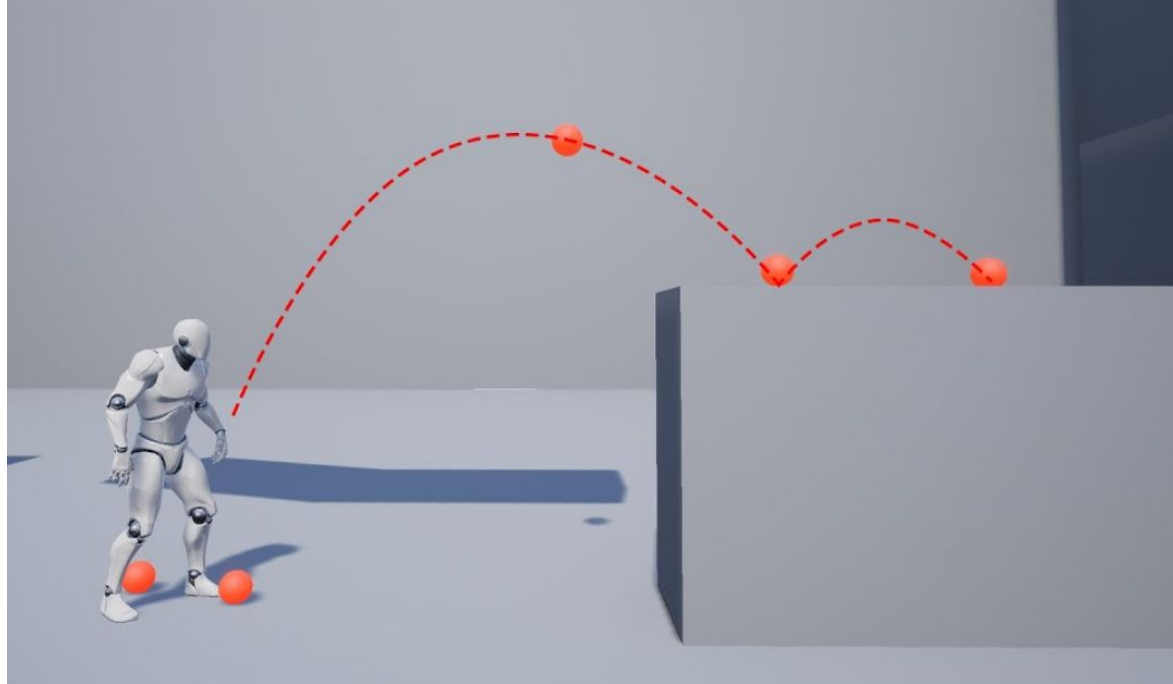
soggetto a fisica

1. I **keyframes** devono essere costruiti con una **struct**
2. Rendere **tunabile** la durata massima della simulazione
3. Usare **l'interpolazione** per posizionare l'oggetto durante tutta la durata della riproduzione
4. Creare un sistema per campionare la fisica di un oggetto per una durata di tempo con una certa **frequenza di campionamento**. I dati campionati devono essere utilizzabili per la riproduzione della simulazione fisica





Balistica



La **balistica** può essere utilizzata per **lanciare** oggetti con/senza gravità per colpire punti fermi o oggetti in movimento, grazie all'utilizzo di **sistemi di equazioni**



Balistica

Equazione moto uniformemente accelerato

$$\vec{P}(t) = \vec{P}_0 + \vec{V}_0 t + \frac{1}{2} \vec{A} t^2$$

t = istante di tempo

P_t = posizione al tempo t

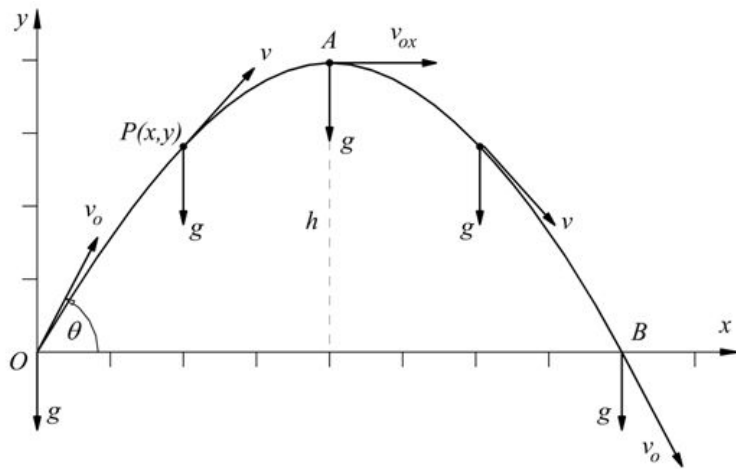
P_0 = posizione iniziale (tempo = 0)

V_0 = velocità iniziale (tempo = 0)

A = accelerazione

Per la balistica consideriamo l'**accelerazione di gravità** $\rightarrow g = -9.81 \text{ m/s}^2$

L'accelerazione agisce durante tutto il moto ed è **costante**, quello che varia è la **velocità verticale**. In assenza altre forze come la resistenza dell'aria, anche la **velocità orizzontale** rimane costante per tutto il percorso



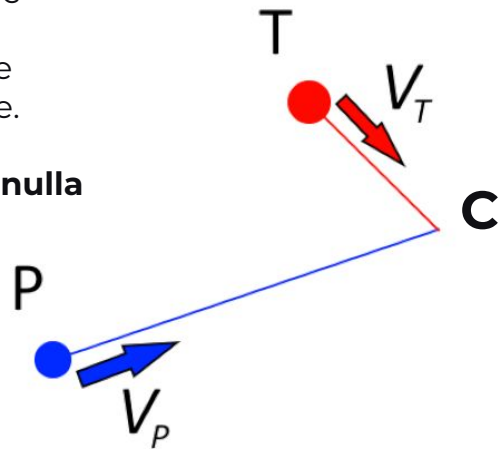


Balistica

Caso 1 - Bersaglio mobile, senza gravità

Consideriamo un caso in cui abbiamo un **bersaglio T** da colpire con un **proiettile P**. L'intensità della **velocità del proiettile** è conosciuta ma dobbiamo calcolare la sua direzione per riuscire a colpire il bersaglio. La **gravità non influenza** nessuno dei due. **Entrambi** si muovono in **moto rettilineo uniforme**, quindi il **termine quadratico sparisce** in quanto l'accelerazione risulta **nulla**

$$\begin{cases} \vec{P}_P(t) = \vec{P}_{P_0} + \vec{V}_P t & \text{moto proiettile P} \\ \vec{P}_T(t) = \vec{P}_{T_0} + \vec{V}_T t & \text{moto bersaglio T} \end{cases}$$



Questo sistema di *due* equazioni ha *due* incognite ed è dunque *determinato* quando

1. L'**intervallo di tempo Δt** impiegato per raggiungere il punto di incontro **C** è lo stesso da entrambi gli oggetti in moto
2. La **posizione** del proiettile **P** e la **posizione** del bersaglio **T** sono uguali dopo **Δt** secondi dall'inizio del moto



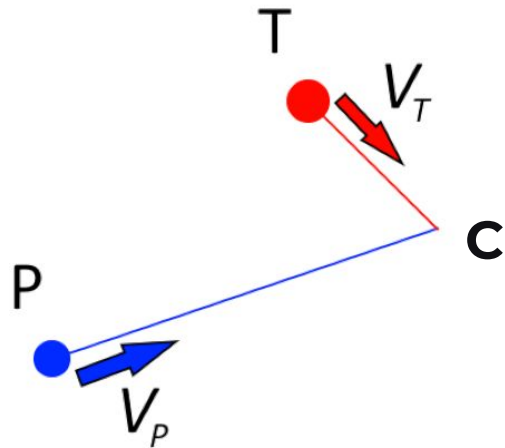
Balistica

Caso 1 - Bersaglio mobile, senza gravità

Procedimento per la risoluzione

1. **Eguagliare** la posizione del proiettile e del bersaglio al medesimo tempo t per avere un sistema determinato e risolvibile (P e T raggiungono C)
2. Calcolare le **componenti della velocità del proiettile** in funzione del tempo impiegato
3. Ottenere un'**equazione quadratica** in funzione dell'*inverso del tempo impiegato*
4. Controllare il *delta* dell'equazione quadratica
5. Risolvere l'equazione tenendo conto del **tempo positivo minore**

Ricordarsi di considerare casi particolari perchè non sempre c'è soluzione!





Balistica

Caso 1 - Bersaglio mobile, senza gravità

Stessa posizione finale con lo stesso **tempo** impiegato

Calcolo le **componenti della velocità del proiettile** in funzione del tempo impiegato (assumendo che Δt non sia zero - si può controllare vedendo che le posizioni iniziali di P e T siano diverse)

$$\begin{cases} X_{P_0} + V_{P_X} \Delta t = X_{T_0} + V_{T_X} \Delta t \\ Y_{P_0} + V_{P_Y} \Delta t = Y_{T_0} + V_{T_Y} \Delta t \\ Z_{P_0} + V_{P_Z} \Delta t = Z_{T_0} + V_{T_Z} \Delta t \end{cases}$$

$$\begin{cases} V_{P_X} = \frac{X_{T_0} - X_{P_0}}{\Delta t} + V_{T_X} \\ V_{P_Y} = \frac{Y_{T_0} - Y_{P_0}}{\Delta t} + V_{T_Y} \\ V_{P_Z} = \frac{Z_{T_0} - Z_{P_0}}{\Delta t} + V_{T_Z} \end{cases}$$



Balistica

Caso 1 - Bersaglio mobile, senza gravità

k inverso del tempo impiegato
 Δi spostamento sull'asse i -esimo

Sostituisco ed ottengo le **componenti della velocità del proiettile** in funzione di k

$$\begin{cases} k = \frac{1}{\Delta t} \\ \Delta i_0 = i_{T_0} - i_{P_0} \end{cases}$$

$$\begin{cases} V_{P_X} = \Delta X_0 k + V_{T_X} \\ V_{P_Y} = \Delta Y_0 k + V_{T_Y} \\ V_{P_Z} = \Delta Z_0 k + V_{T_Z} \end{cases}$$



Balistica

Caso 1 - Bersaglio mobile, senza gravità

Conosciamo **l'intensità** della velocità del proiettile ma **non le sue componenti**. L'equazione della magnitudo al quadrato ci viene data da **Pitagora**

$$V_{P_X}^2 + V_{P_Y}^2 + V_{P_Z}^2 = \|V_P\|^2$$

Sostituendo le componenti della velocità con le equazioni in funzione di **k** otteniamo

$$\begin{aligned} &\Delta X_0^2 k^2 + 2V_{T_X} \Delta X_0 k + V_{T_X}^2 + \\ &\Delta Y_0^2 k^2 + 2V_{T_Y} \Delta Y_0 k + V_{T_Y}^2 + \\ &\Delta Z_0^2 k^2 + 2V_{T_Z} \Delta Z_0 k + V_{T_Z}^2 = \|V_P\|^2 \end{aligned}$$



Balistica

Caso 1 - Bersaglio mobile, senza gravità

$$\Delta X_0^2 k^2 + 2V_{T_X} \Delta X_0 k + V_{T_X}^2 + \Delta Y_0^2 k^2 + 2V_{T_Y} \Delta Y_0 k + V_{T_Y}^2 + \Delta Z_0^2 k^2 + 2V_{T_Z} \Delta Z_0 k + V_{T_Z}^2 = \|V_P\|^2$$

raggruppando i termini per ottenere un'equazione quadratica in forma **$ak^2 + bk + c = 0$**

$$(\Delta X_0^2 + \Delta Y_0^2 + \Delta Z_0^2)k^2 + 2(V_{T_X} \Delta X_0 + V_{T_Y} \Delta Y_0 + V_{T_Z} \Delta Z_0)k + V_{T_X}^2 + V_{T_Y}^2 + V_{T_Z}^2 - \|V_P\|^2 = 0$$

Le componenti dell'equazione quadratica ottenute sono

$$\begin{cases} a = \Delta X_0^2 + \Delta Y_0^2 + \Delta Z_0^2 \\ b = 2(V_{T_X} \Delta X_0 + V_{T_Y} \Delta Y_0 + V_{T_Z} \Delta Z_0) \\ c = V_{T_X}^2 + V_{T_Y}^2 + V_{T_Z}^2 - \|V_P\|^2 \end{cases}$$

Notare che tutti questi valori sono noti, pertanto è possibile risolvere l'equazione



Balistica

Caso 1 - Bersaglio mobile, senza gravità

$$\begin{cases} a = \Delta X_0^2 + \Delta Y_0^2 + \Delta Z_0^2 \\ b = 2(V_{T_X}\Delta X_0 + V_{T_Y}\Delta Y_0 + V_{T_Z}\Delta Z_0) \\ c = V_{T_X}^2 + V_{T_Y}^2 + V_{T_Z}^2 - \|V_P\|^2 \end{cases}$$

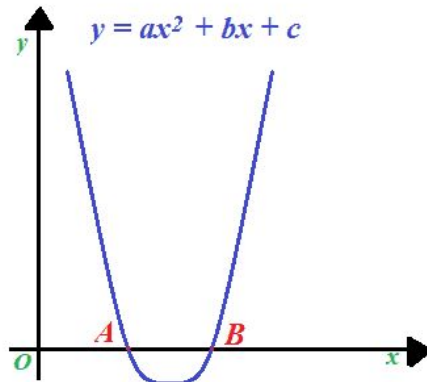
riscrivendo i valori in termini vettoriali otteniamo

$$\begin{cases} a = \left\| \overrightarrow{P_{T_0}} - \overrightarrow{P_{P_0}} \right\|^2 \\ b = 2\overrightarrow{V_T} \cdot (\overrightarrow{P_{T_0}} - \overrightarrow{P_{P_0}}) \\ c = \|\overrightarrow{V_T}\|^2 - \|\overrightarrow{V_P}\|^2 \end{cases}$$

la distanza al quadrato dei punti iniziali

il doppio del **dot product** tra la velocità del bersaglio e il vettore dal punto iniziale del proiettile a quello del bersaglio

la differenza tra il quadrato dell'intensità della velocità del bersaglio e del proiettile





Balistica

Caso 1 - Bersaglio mobile, senza gravità

Soluzioni possibili: **nessuna, una o due**

$$k = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Calcolo il delta: l'equazione ha **risultato**
se il **delta è maggiore o uguale a 0**

$$\Delta = b^2 - 4ac \geq 0$$

Sostituisco **a**, **b** e **c** trovando uno o due valori di **k**

$$k = \frac{-2\vec{V}_T \cdot (\vec{P}_{T_0} - \vec{P}_{P_0}) \pm \sqrt{[2\vec{V}_T \cdot (\vec{P}_{T_0} - \vec{P}_{P_0})]^2 - 4\left\|\vec{P}_{T_0} - \vec{P}_{P_0}\right\|^2 (\|V_T\|^2 - \|V_P\|^2)}}{2\left\|\vec{P}_{T_0} - \vec{P}_{P_0}\right\|^2}$$

Inverto k e tengo il **valore positivo minore**

$$\Delta t = \frac{1}{k}$$



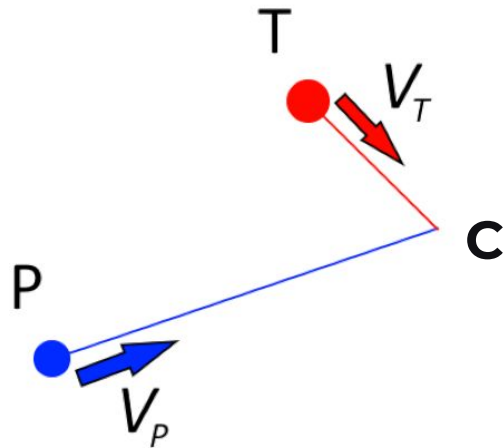
Balistica

Caso 1 - Bersaglio mobile, senza gravità

Conoscendo il Δt possiamo calcolare le **componenti della velocità** che il proiettile deve avere all'inizio

$$\begin{cases} V_{P_X} = \frac{X_{T_0} - X_{P_0}}{\Delta t} + V_{T_X} \\ V_{P_Y} = \frac{Y_{T_0} - Y_{P_0}}{\Delta t} + V_{T_Y} \\ V_{P_Z} = \frac{Z_{T_0} - Z_{P_0}}{\Delta t} + V_{T_Z} \end{cases}$$

ovvero, in forma vettoriale
$$\vec{V}_P = \frac{\vec{P}_{T_0} - \vec{P}_{P_0}}{\Delta t} + \vec{V}_T$$





Balistica

Caso 1 - Bersaglio mobile, senza gravità

Irrisolubile quando il delta Δ è negativo perchè

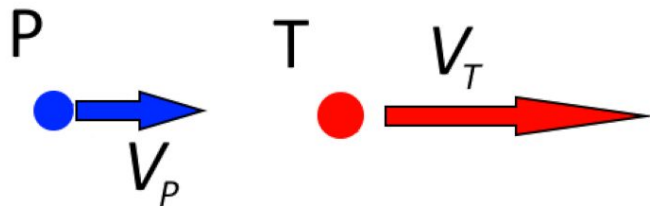
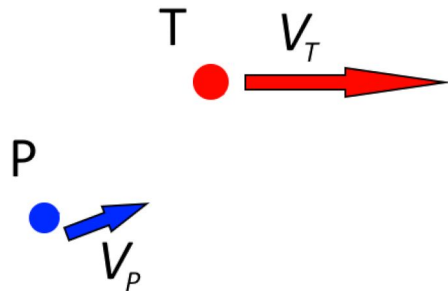
1. Bersaglio troppo **veloce**
2. Distanza **iniziale** troppo elevata

$$[2\vec{V}_T \cdot (\vec{P}_{T_0} - \vec{P}_{P_0})]^2 - 4\left\|\vec{P}_{T_0} - \vec{P}_{P_0}\right\|^2 (\|\vec{V}_T\|^2 - \|\vec{V}_P\|^2)$$

Delta Δ positivo ma soluzione Δt negativo significa che il bersaglio T si sta allontanando dal proiettile P con la **stessa direzione con velocità sufficientemente alta**

$$-2\vec{V}_T \cdot (\vec{P}_{T_0} - \vec{P}_{P_0})$$

La soluzione è negativa perché l'istante di tempo in cui si incontrano è **nel passato** rispetto all'istante iniziale





Balistica

ESERCIZI

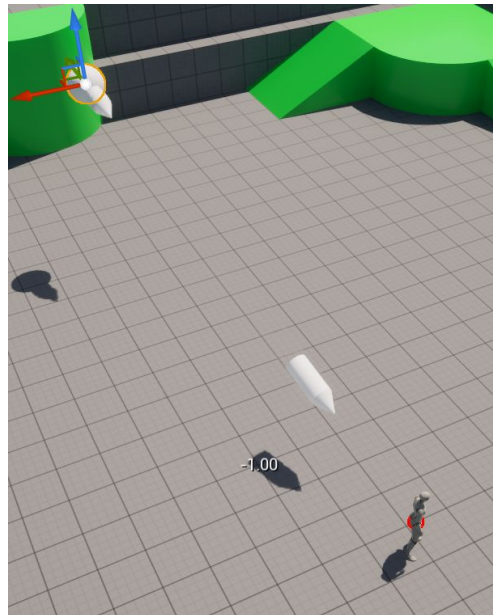
Torretta spara proiettili

Creare uno **spawner di proiettili** con le seguenti caratteristiche

1. I proiettili **non** devono essere soggetti a **gravità** e devono colpire il player
2. Il player deve venir colpito **se** durante il ciclo vitale del proiettile quest'ultimo **non cambia direzione o intensità** di movimento
3. Il proiettile deve **mantenere la direzione** della sua velocità iniziale per tutto il proprio ciclo vitale
4. Mostrare il previsto punto d'impatto con il giocatore con una **sfera di debug**

E' consigliato utilizzare il **Projectile Movement Component** con *Initial Velocity in Local Space* settata a *false* e *Projectile Gravity Scale* a 0

Per impostare la velocità del proiettile si può direttamente modificare il vettore **ProjectileComponent->Velocity**





Balistica

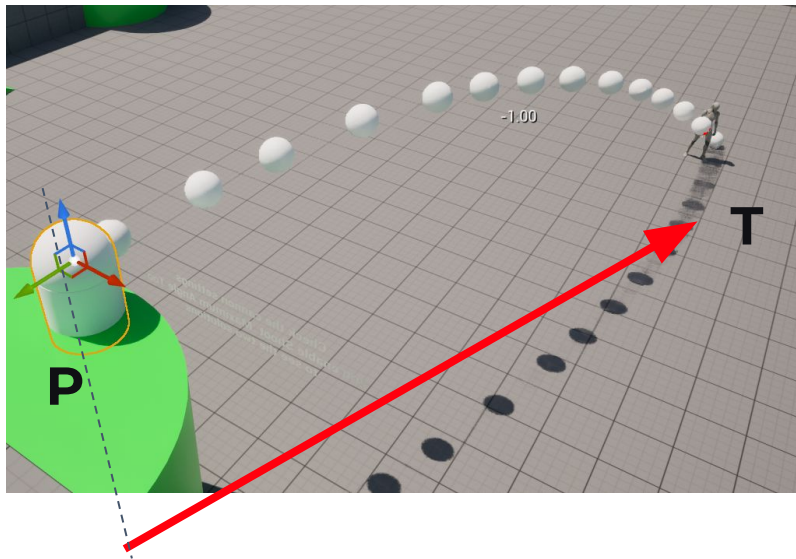
Caso 2 - Bersaglio fermo, gravità attiva

Il proiettile P deve raggiungere il bersaglio T prendendo la **direzione** corretta

Sul piano equatoriale, la **direzione** è facilmente ottenibile conoscendo i punti P_0 e T

$$\frac{(X_T - X_{P_0}, Y_T - Y_{P_0})}{\sqrt{(X_T - X_{P_0})^2 + (Y_T - Y_{P_0})^2}}$$

Questo vettore normalizzato lo utilizzeremo più avanti quando tutte le incognite saranno risolte





Balistica

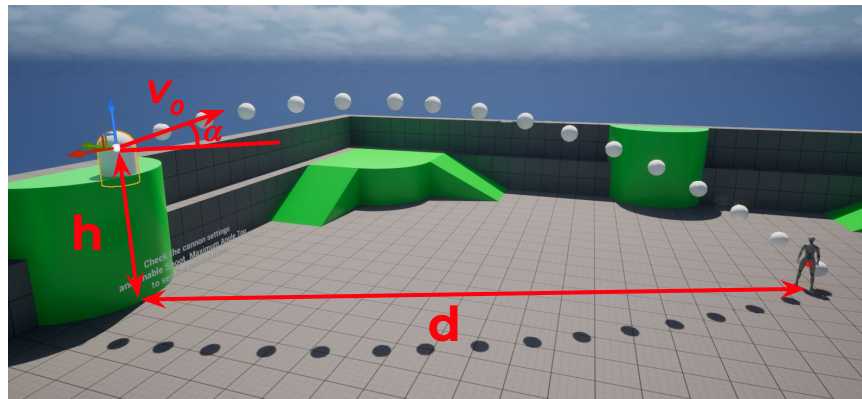
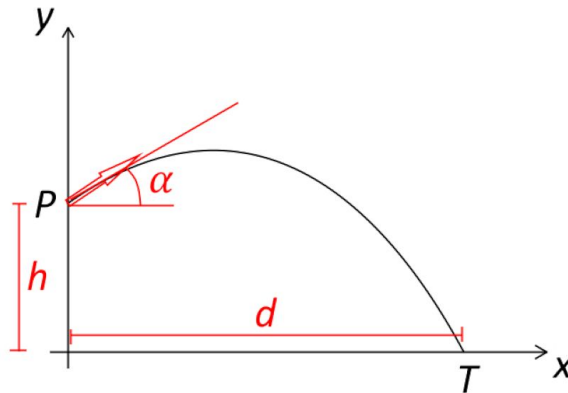
Caso 2 - Bersaglio fermo, gravità attiva

Consideriamo il piano parallelo all'asse Z, che passa sia per il punto iniziale del proiettile P_0 sia per il punto finale T

I dati conosciuti sono

- h : distanza sull'**asse Z** tra P_0 e T
- d : distanza sul **piano XY** tra P_0 e T
- g : accelerazione di gravità (-981 cm/s²)

Per sapere la **direzione** verso la quale dobbiamo sparare il proiettile dobbiamo calcolare α , ovvero l'angolo di tiro rispetto al piano XY





Balistica

Caso 2 - Bersaglio fermo, gravità attiva

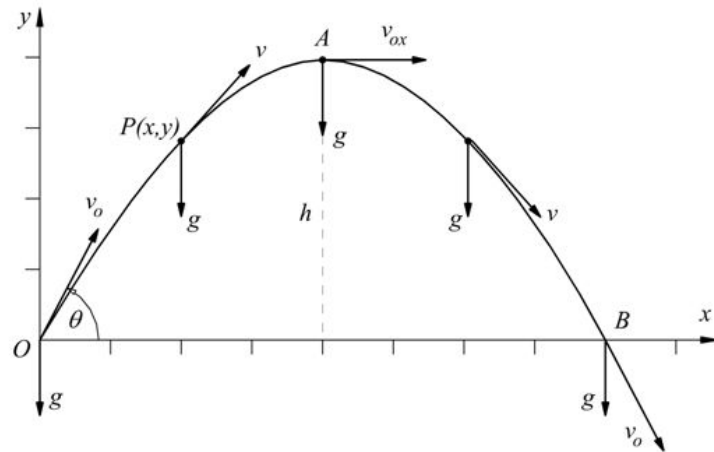
Equazione del moto uniformemente accelerato

$$\vec{P}(t) = \vec{P}_0 + \vec{V}_0 t + \frac{1}{2} \vec{A} t^2$$

Dopo un intervallo di tempo Δt il proiettile raggiunge la posizione del bersaglio $\mathbf{T}$

$$\vec{T} = \vec{P}_0 + \vec{V}_0 \Delta t + \frac{1}{2} \vec{g} \Delta t^2$$

e l'accelerazione di **gravità** è l'**unica accelerazione** che agisce (non consideriamo resistenza dell'aria, vento o altri fattori)





Balistica

Caso 2 - Bersaglio fermo, gravità attiva

L'accelerazione di gravità agisce solo sull'**asse verticale Z**, che però d'ora in avanti scriviamo come **Y** del piano su cui si muove il proiettile durante il moto. **X** invece è un asse sul piano **XY**

$$\begin{cases} X_{P_0} + V_{0_X} \Delta t = X_T \\ Y_{P_0} + V_{0_Y} \Delta t + \frac{1}{2} g \Delta t^2 = Y_T \end{cases}$$

Riscrivendo la prima equazione secondo Δt si ottiene
$$\Delta t = \frac{X_T - X_{P_0}}{V_{0_X}}$$

Scomponendo la velocità nelle sue **componenti** su X e Y arriviamo ad avere

$$\begin{cases} V_{0_X} = \|V_0\| \cos(\alpha) \\ V_{0_Y} = \|V_0\| \sin(\alpha) \end{cases} \quad \text{e quindi sostituendo} \quad \Delta t = \frac{X_T - X_{P_0}}{\|V_0\| \cos(\alpha)}$$



Balistica

Caso 2 - Bersaglio fermo, gravità attiva

La differenza tra il punto iniziale e il punto finale sull'asse X è nota quindi la sostituiamo

$$\Delta t = \frac{X_T - X_{P_0}}{\|V_0\| \cos(\alpha)} = \frac{d}{\|V_0\| \cos(\alpha)} \quad \text{con } \mathbf{d} \text{ distanza 2D tra } \mathbf{P}_0 \text{ e } \mathbf{T}$$

Riprendiamo l'equazione anche su Y e sostituiamo il Δt per ottenere

$$Y_{P_0} + V_{0Y} \frac{d}{\|V_0\| \cos(\alpha)} + \frac{1}{2} g \left(\frac{d}{\|V_0\| \cos(\alpha)} \right)^2 = Y_T$$

e sostituendo anche la componente Y della velocità $\mathbf{V}_0$ si ottiene

$$Y_{P_0} + \|V_0\| \sin(\alpha) \frac{d}{\|V_0\| \cos(\alpha)} + \frac{1}{2} g \left(\frac{d}{\|V_0\| \cos(\alpha)} \right)^2 = Y_T$$



Balistica

Caso 2 - Bersaglio fermo, gravità attiva

L'equazione ottenuta ha solo α come incognita

$$Y_{P_0} + \|V_0\| \sin(\alpha) \frac{d}{\|V_0\| \cos(\alpha)} + \frac{1}{2} g \left(\frac{d}{\|V_0\| \cos(\alpha)} \right)^2 = Y_T$$

$$Y_{P_0} + \frac{d \|V_0\| \sin(\alpha)}{\|V_0\| \cos(\alpha)} + \frac{1}{2} g \frac{d^2}{\|V_0\|^2 \cos^2(\alpha)} = Y_T$$

$$Y_{P_0} - Y_T + d \cdot \tan(\alpha) + \frac{gd^2}{2\|V_0\|^2 \cos^2(\alpha)} = 0$$

$$h + d \cdot \tan(\alpha) + \frac{gd^2}{2\|V_0\|^2 \cos^2(\alpha)} = 0$$

dato che h la conosciamo
come dato di partenza



Balistica

Caso 2 - Bersaglio fermo, gravità attiva

Avendo **coseno** e **tangente**, dobbiamo convertire uno dei due nell'altro per avere un'equazione quadratica risolvibile

Partiamo dalla prima relazione fondamentale

$$\cos^2(\alpha) + \sin^2(\alpha) = 1$$

Dividiamo tutto per **$\cos^2(\alpha)$** assumendo che α **non sia $\pm 90^\circ$** (che ci va bene perchè vorrebbe dire sparare il proiettile in verticale)

$$1 + \tan^2(\alpha) = \frac{1}{\cos^2(\alpha)}$$

Sostituendo nell'equazione ottenuta prima abbiamo quindi

$$h + d \cdot \tan(\alpha) + \frac{gd^2(1 + \tan^2(\alpha))}{2\|V_0\|^2} = 0$$



Balistica

Caso 2 - Bersaglio fermo, gravità attiva

Cerchiamo di sistemare l'equazione per avere una forma quadratica $\mathbf{ak^2 + bk + c = 0}$

$$h + d \cdot \tan(\alpha) + \frac{gd^2(1 + \tan^2(\alpha))}{2\|V_0\|^2} = 0$$

$$h + d \cdot \tan(\alpha) + \frac{gd^2}{2\|V_0\|^2} + \frac{gd^2}{2\|V_0\|^2} \tan^2(\alpha) = 0$$

$$\frac{gd^2}{2\|V_0\|^2} \tan^2(\alpha) + d \cdot \tan(\alpha) + \frac{gd^2}{2\|V_0\|^2} + h = 0$$



Balistica

Caso 2 - Bersaglio fermo, gravità attiva

Assumendo che la **velocità iniziale V_0 non sia nulla** moltiplichiamo tutto per $2\|V_0\|^2$

$$\frac{gd^2}{2\|V_0\|^2} \tan^2(\alpha) + d \cdot \tan(\alpha) + \frac{gd^2}{2\|V_0\|^2} + h = 0$$

$$gd^2 \tan^2(\alpha) + 2d\|V_0\|^2 \tan(\alpha) + gd^2 + 2h\|V_0\|^2 = 0$$

e assumendo che **d** (distanza orizzontale tra punto iniziale e punto finale) non sia nullo, dividiamo tutto per **$2d$** ottenendo

$$\frac{gd}{2} \tan^2(\alpha) + \|V_0\|^2 \tan(\alpha) + \frac{gd}{2} + \frac{h\|V_0\|^2}{d} = 0$$



Balistica

Caso 2 - Bersaglio fermo, gravità attiva

L'equazione quadratica è dunque

$$\frac{gd}{2} \tan^2(\alpha) + \|V_0\|^2 \tan(\alpha) + \frac{gd}{2} + \frac{h\|V_0\|^2}{d} = 0$$

Dove l'incognita è **$\tan^2\alpha$** e i coefficienti sono

$$a = \frac{gd}{2}$$

$$b = \|V_0\|^2$$

$$c = \frac{gd}{2} + \frac{h\|V_0\|^2}{d}$$



Balistica

Caso 2 - Bersaglio fermo, gravità attiva

Troviamo le **soluzioni** con la formula della soluzione **analitica** delle equazioni di secondo grado

$$\tan(\alpha) = \frac{-\|V_0\|^2 \pm \sqrt{(\|V_0\|^2)^2 - 4\left(\frac{gd}{2}\right)\left(\frac{gd}{2} + \frac{h\|V_0\|^2}{d}\right)}}{2\left(\frac{gd}{2}\right)}$$

$$\tan(\alpha) = \frac{-\|V_0\|^2 \pm \sqrt{\|V_0\|^4 - g(gd^2 + 2h\|V_0\|^2)}}{gd}$$

Le soluzioni esistono se il **delta** (espressione sotto la radice quadrata) non è negativo

$$\Delta = \|V_0\|^4 - g(gd^2 + 2h\|V_0\|^2) \geq 0$$



Balistica

Caso 2 - Bersaglio fermo, gravità attiva

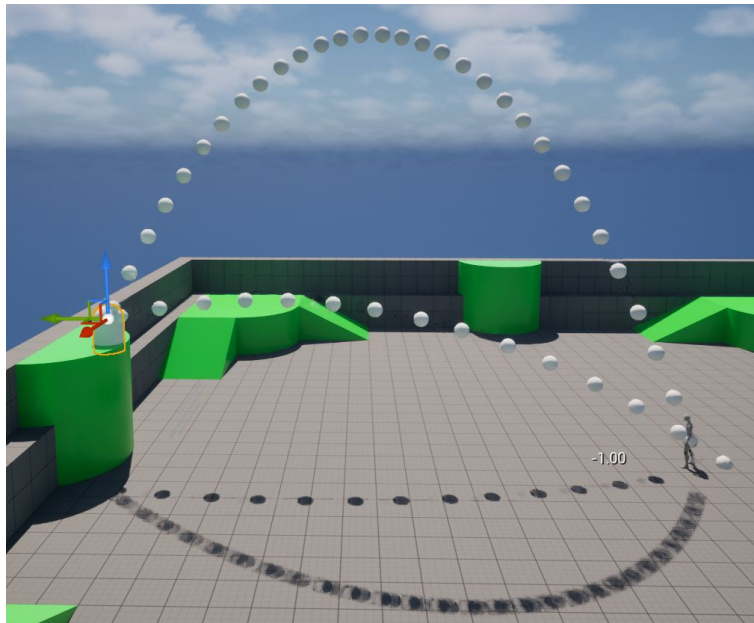
Facendo **l'arcotangente** delle due soluzioni otteniamo i **due possibili angoli** che risultano in una traiettoria che porta a colpire il bersaglio T. Di solito si prende l'**angolo minore** ma si potrebbe considerare anche quello maggiore a seconda del design del gioco.

Dall'angolo ottenuto si può **ricostruire la direzione** che deve avere la velocità in

$$V_{XY} = \|V_0\| \cos(\alpha)$$

$$V_Z = \|V_0\| \sin(\alpha)$$

La componente su **XY** va moltiplicata per il **vettore normalizzato calcolato all'inizio** mentre la **componente Z** va **semplicemente utilizzata** come componente Z della velocità





Balistica

Caso 2 - Bersaglio fermo, gravità attiva

La **gittata** (distanza massima) si può calcolare considerando il **delta** dell'equazione quadratica **pari a 0**. Quando il delta è zero le soluzioni convergono in una sola perchè esiste un **solo angolo**

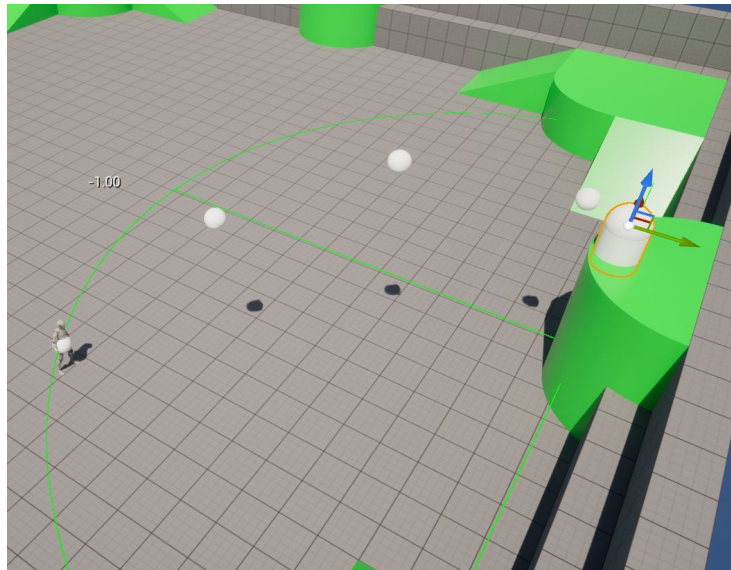
$$\|V_0\|^4 - g(gd^2 + 2h\|V_0\|^2) = 0$$

Riscrivendo con **d** come incognita si ottiene

$$d = \sqrt{\frac{\|V_0\|^4}{g^2} - \frac{2h\|V_0\|^2}{g}}$$

Se il bersaglio è all'altezza del punto di spawn del proiettile, il termine per **h** si annulla diventando

$$d = \frac{\|V_0\|^2}{g}$$





Balistica

ESERCIZI

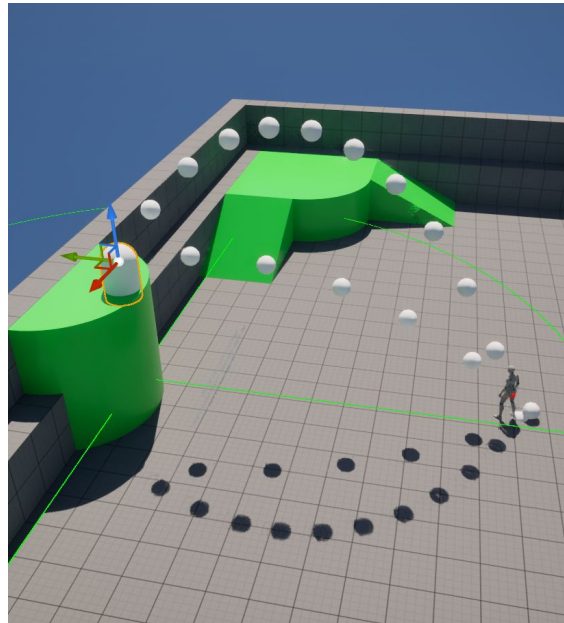
Torretta cannone

Creare uno **spawner di palle di cannone** con le seguenti caratteristiche

1. Le palle di cannone **devono** essere soggette a **gravità** (tunabile) e devono colpire il player
2. Il player deve venir colpito **se sta fermo** durante il ciclo vitale della palla di cannone
3. Il proiettile deve cambiare direzione solo per effetto della gravità, ma la **direzione iniziale** va impostata **una sola volta**
4. Mostrare il previsto **punto d'impatto** con il giocatore e **gittata** con una **forme di debug**
5. *Nice to have*: sparare una **seconda palla** di cannone usando l'**angolo massimo** invece che quello minimo

E' consigliato utilizzare il **Projectile Movement Component** con *Initial Velocity in Local Space* uguale a *false* e *Projectile Gravity Scale* pari alla **gravità tunabile**

Per impostare la velocità del proiettile si può modificare il vettore **ProjectileComponent->Velocity**





Domande?

